InvisiGuard - Intelligent System for Predicting Cyber Threats in Real Time

Project ID: 25-26J-062

Name	IT Number
NAWARATHNA M N I M	IT22343116
PALLEWATHTHA P A	IT22133304
NAWARATHNA N P D T	IT22117496
NETHKALUM G M H	IT22562906

BSc (Hons) Degree in Information Technology Specializing in Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology Faculty of Computing

April 2026

Declaration

We declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. We Remove inappropriate term 6 Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books)

Name	Signature	Date
NAWARATHNA M N I M IT22343116		
PALLEWATHTHA P.A- IT22133304		
NAWARATHNA N P D T IT22117496		
NETHKALUM G.M.H- IT22562906		

Signature of Supervisor: Mr. Amila Senarathne Date

ABSTRACT

Traditional cybersecurity systems mainly rely on signature-based and rule-based detection methods. These methods work well for identifying known threats but struggle to detect evolving, stealthy, and new attacks. This creates a significant challenge in modern cybersecurity. Organizations must protect themselves against both known attacks and zero-day threats across various layers of digital infrastructure. To tackle this issue, this project introduces a Real Time Cyber Threats Predicting System that employs machine learning and deep learning techniques. It predicts cyber threats by analyzing multiple security data sources. The project consists of four main components: predicting known attacks using system call patterns, predicting known attacks through user behavioral patterns, predicting zero-day attacks based on network traffic patterns, and predicting known attacks using Windows endpoint log patterns. Together, these components form a layered and behavior-based cybersecurity framework.

The system call component examines process-level activity by turning system events into structured traces. It uses a staged detection process with a Random Forest model and an RNN-based sequential model. The user behavior component applies an LSTM-based User and Entity Behavior Analytics model to spot unusual and harmful user activities by looking at temporal behavior patterns. The network traffic component focuses on predicting zero-day attacks with a combined anomaly detection framework. This framework merges with an Autoencoder, Isolation Forest, and a logistic regression meta-classifier. The endpoint log component reviews Sysmon and PowerShell logs using a feature-rich representation and an XGBoost multiclass classifier. This analysis helps identify known attack categories such as credential dumping, defense evasion, lateral movement, and privilege escalation. These specific approaches enable the system to gather threat evidence from different layers of user, process, host, and network activity.

The experimental results show strong performance across the project's main components. The user behavior model achieved excellent classification results. The hybrid network traffic model surpassed standalone anomaly detection models, and the endpoint log component demonstrated strong multiclass attack prediction performance. Overall, the results confirm that combining multiple machine learning models across diverse cybersecurity data sources can greatly enhance proactive threat prediction and support real-time security monitoring. Therefore, this project offers a practical and intelligence-driven cybersecurity framework that moves beyond traditional reactive defense, laying a solid foundation for the future development of integrated predictive security systems.

Acknowledgment

We would like to express our sincere appreciation to all individuals who contributed to the successful completion of our research project titled “InvisiGuard – Intelligent System for Predicting Cyber Threats in Real Time.”

First and foremost, we extend our deepest gratitude to our Supervisor, Mr. Amila Senarathne, for his invaluable guidance, continuous support, and insightful feedback throughout the course of this research. His expertise and commitment were instrumental in shaping the direction and quality of this work.

We also wish to express our profound gratitude to our Co-supervisor, Mr. Deemantha Siriwardana, for his constructive suggestions, professional guidance, and encouragement, which significantly enhanced the academic and practical aspects of this project.

Furthermore, we acknowledge the Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology (SLIIT) for providing the necessary academic environment, technical resources, and institutional support required to conduct this research effectively.

Finally, we extend our sincere thanks to all lecturers, peers, and staff members of SLIIT whose direct and indirect support contributed to the successful completion of this research. This work reflects collective effort, shared knowledge, and the collaborative academic environment we deeply value.

Table of Contents

Acknowledgment 4
1. Introduction 7
1.1 Background Literature 8
1.2 Research Gap 11
1.3 Research Problem 19
2. Methodology 24
2.1 Introduction to Methodology 25
2.2 Requirement Gathering 25
2.3 Stakeholder Analysis 26
2.4 System Architecture Overview 27
2.5 Functional Requirements 29
2.6 Non-Functional Requirements 33
2.7 Commercialization and Deployment Considerations 35
2.8 Tools and Technologies 36
3. Results and Discussion 40
3.1 Research Finding 43
3.2 Overall Components Findings 45
4.Summary of Each Student's contribution 46
5.Conclusion 47
6. References 48

LIST OF ABBREVIATION

Abbreviation	**Description**
AI	Artificial Intelligence
ML	Machine Learning
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
SVM	Support Vector Machine
SIEM	Security Information and Event Management
SOC	Security Operations Center
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
EDR	Endpoint Detection and Response
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IPC	Inter-Process Communication
RF	Random Forest
RNN	Recurrent Neural Network

1. Introduction

The digital transformation of modern enterprises has fundamentally altered the cybersecurity landscape, creating an environment where traditional defense mechanisms are increasingly inadequate. As organizations migrate critical infrastructure to cloud platforms and adopt distributed work models, the attack surface has expanded exponentially, providing adversaries with numerous entry points for exploitation. Contemporary cyber threats have evolved from simple, signature-based attacks to sophisticated, multi-vector campaigns that leverage advanced persistent threats, zero-day vulnerabilities and insider threats to compromise organizational security.

Traditional cybersecurity systems predominantly rely on signature-based detection methods, which operate on the principle of matching observed behaviors against known attack patterns. While effective against documented threats, this reactive approach fundamentally fails when confronted with novel attack vectors, zero-day exploits, or sophisticated insider threats that operate within the boundaries of normal system behavior. The average time to detect a security breach remains alarmingly high, with studies indicating that organizations take an average of 207 days to identify a breach and an additional 73 days to contain it. This detection latency provides adversaries with extended windows of opportunity to exfiltrate sensitive data, establish persistence mechanisms, and cause substantial organizational damage. The increasing sophistication of cyberattacks necessitates a paradigm shift from reactive detection to proactive prediction. Modern threat actors employ advanced techniques including polymorphic malware, living-off-the-land tactics, and behavioral mimicry to evade traditional security controls. Furthermore, the rise of insider threats whether malicious or inadvertent presents unique challenges, as these actors possess legitimate credentials and intimate knowledge of organizational systems, making their activities difficult to distinguish from normal operations.[1]

This research addresses these critical challenges by proposing InvisiGuard, an intelligent, multi-layered cybersecurity framework that leverages machine learning to predict both known and zero-day attacks across multiple data dimensions. Unlike conventional systems that analyze single data sources in isolation, InvisiGuard integrates four complementary detection components: system call analysis, user behavior monitoring, endpoint log inspection, and network traffic analysis. By correlating insights across these diverse data streams and employing hybrid machine learning architectures that combine supervised and unsupervised learning techniques, the system provides comprehensive, real-time threat intelligence capable of identifying both documented attack patterns and previously unseen anomalies.

The significance of this research extends beyond technical innovation to address fundamental operational challenges in enterprise security. By reducing detection latency, minimizing false positive rates, and providing actionable intelligence through integration with the MITRE ATT&CK framework, InvisiGuard enables security operations centers (SOCs) to transition from reactive incident response to proactive threat hunting. The system's unified dashboard consolidates

alerts from multiple detection engines, providing security analysts with a holistic view of organizational security posture and enabling rapid, informed decision-making.

This report presents the design, implementation, and evaluation of the InvisiGuard system, demonstrating how integrated machine learning approaches can transform cybersecurity from a reactive discipline into a proactive, intelligence-driven practice. The following sections provide a comprehensive review of existing literature, identify critical research gaps, articulate the specific research problem, and outline the objectives that guided this investigation.

1.1 Background Literature

The rapid advancement of digital technologies has significantly transformed the way organizations operate, leading to increased dependence on interconnected systems, cloud infrastructures, and large-scale data processing environments. While these advancements have improved operational efficiency and scalability, they have also introduced complex cybersecurity challenges. Over the past decade, researchers and industry professionals have extensively studied various approaches to detect, prevent, and respond to cyber threats. This section reviews the existing literature on traditional cybersecurity mechanisms, machine learning–based threat detection, behavioral analysis techniques, and challenges in modern cybersecurity systems.[2]

Traditional Cybersecurity Approaches

Conventional cybersecurity solutions, such as Intrusion Detection Systems (IDS), Intrusion Prevention Systems (IPS), and Security Information and Event Management (SIEM) platforms, have long been the foundation of organizational security infrastructures. These systems primarily rely on signature-based detection and rule-based mechanisms to identify malicious activities. Signature-based systems compare incoming data against a database of known attack patterns, making them effective in detecting previously identified threats. However, their effectiveness is limited when dealing with unknown or evolving threats, such as zero-day attacks.

Rule-based systems, on the other hand, depend on predefined conditions and thresholds to trigger alerts. While they offer flexibility and customization, they require continuous updates and fine-tuning to remain effective. Studies have shown that these traditional approaches often generate a high number of false positives, leading to alert fatigue among security analysts and reducing the overall efficiency of incident response processes. Additionally, these systems lack the ability to understand contextual and behavioral aspects of user and system activities, making it difficult to detect insider threats and sophisticated attack patterns.

Machine Learning in Cybersecurity

To overcome the limitations of traditional methods, researchers have increasingly explored the application of Artificial Intelligence (AI) and Machine Learning (ML) in cybersecurity. Machine learning techniques enable systems to learn from historical data, identify patterns, and make predictions without being explicitly programmed. This capability makes ML particularly suitable for detecting anomalies and identifying previously unseen threats.

Supervised learning algorithms, such as Decision Trees, Support Vector Machines (SVM), and Random Forests, have been widely used for classifying network traffic and identifying known attack patterns. These models are trained on labeled datasets containing examples of both benign and malicious activities. While supervised learning provides high accuracy for known threats, it requires large volumes of labeled data, which may not always be available in real-world scenarios.

Unsupervised learning techniques, including clustering and anomaly detection algorithms, have been proposed to identify unusual patterns in data without relying on labeled datasets. These methods are particularly useful for detecting zero-day attacks and novel threats. However, they often face challenges in distinguishing between benign anomalies and actual malicious activities, which can result in false alarms.[3]

Recent studies have also explored hybrid approaches that combine supervised and unsupervised learning techniques to improve detection accuracy and reduce false positives. These approaches leverage the strengths of both methods, enabling systems to detect known threats while also identifying new and emerging attack patterns.

Deep Learning and Behavioral Analysis

Deep learning, a subset of machine learning, has gained significant attention in recent years due to its ability to model complex patterns and relationships in large datasets. Techniques such as Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNN) have been widely applied in cybersecurity research for tasks such as intrusion detection, malware classification, and user behavior analysis.

Behavioral analysis has emerged as a critical approach for detecting insider threats and advanced persistent attacks. Unlike traditional methods that focus on static signatures, behavioral analysis examines patterns of user activities, system calls, and network interactions over time. By establishing a baseline of normal behavior, these systems can identify deviations that may indicate malicious intent.

LSTM networks have been shown to be effective in modeling sequential data, making them suitable for analyzing time-series data such as user activity logs and system events. Similarly, CNNs have been used to extract features from structured and unstructured data, enabling more accurate classification of network traffic and system behaviors.

Despite their advantages, deep learning models require significant computational resources and large datasets for training. Additionally, their “black box” nature can make it difficult to interpret the results, which may limit their adoption in critical security environments where transparency and explainability are essential.

Multi-Source Data Correlation

Modern cybersecurity environments generate vast amounts of data from multiple sources, including network traffic, endpoint logs, system calls, and user activities. Analyzing these data sources in isolation can lead to incomplete or inaccurate threat detection. As a result, researchers have emphasized the importance of multi-source data correlation in improving the effectiveness of cybersecurity systems.

SIEM platforms have traditionally been used to aggregate and correlate data from different sources. However, their reliance on rule-based correlation limits their ability to detect complex and coordinated attacks. Recent research has focused on integrating machine learning techniques into data correlation processes to enhance detection capabilities.

By combining insights from multiple data sources, advanced systems can identify patterns that may not be visible when analyzing individual datasets. This approach enables more accurate detection of sophisticated attacks, such as multi-stage intrusions and insider threats, which often involve subtle and coordinated actions across different layers of the system.

Challenges in Modern Cybersecurity Systems

Despite significant advancements in cybersecurity technologies, several challenges remain. One of the primary challenges is the availability and quality of data. Machine learning models require large volumes of high-quality data for training, but obtaining labeled datasets for cybersecurity applications can be difficult due to privacy concerns and the dynamic nature of cyber threats.

Another challenge is the high rate of false positives generated by detection systems. Excessive false alerts can overwhelm security teams and reduce their ability to respond effectively to real threats. Balancing detection accuracy and false positive rates remains a key research focus.

Scalability is also a major concern, as modern networks generate massive amounts of data that must be processed in real time. Ensuring that detection systems can handle high data volumes without compromising performance is critical for practical deployment.

Additionally, the evolving nature of cyber threats requires systems to be adaptive and continuously updated. Static models and rule-based systems may quickly become outdated, highlighting the need for dynamic and self-learning approaches.

Summary and Research Gap

The reviewed literature highlights the limitations of traditional cybersecurity approaches and the growing importance of machine learning and behavioral analysis in threat detection. While existing solutions have made significant progress, they often focus on specific aspects of cybersecurity, such as network-based detection or endpoint monitoring, without providing a unified and comprehensive solution.

Furthermore, many systems lack the ability to effectively correlate data from multiple sources and provide real-time predictive insights. This gap emphasizes the need for an integrated framework that combines multiple analytical techniques to deliver accurate, scalable, and proactive threat detection.

In response to these challenges, the proposed system, InvisiGuard, aims to address the identified research gaps by integrating supervised learning, deep learning, and anomaly detection techniques within a unified architecture. By leveraging multi-source data and centralized intelligence, the system seeks to enhance the detection of known, insider, and zero-day threats while providing real-time insights and reducing false positives.[4]

1.2 Research Gap

Despite substantial progress in cybersecurity research, several critical gaps persist in current threat detection approaches:

Fragmented Detection Approaches

Existing research predominantly focuses on individual data sources in isolation network traffic, system logs, or user behavior without comprehensive integration. While specialized systems demonstrate effectiveness within their domains, adversaries increasingly employ multi-stage attacks that span multiple layers of the technology stack. Current systems lack the holistic visibility necessary to detect sophisticated attack campaigns that carefully distribute malicious activities across different data dimensions to evade single-source detection mechanisms.

Modern Advanced Persistent Threat (APT) campaigns exemplify this challenge. A typical APT attack begins with initial compromise through spear-phishing (detectable through email analysis), followed by credential harvesting (visible in system call patterns), lateral movement (observable in network traffic), privilege escalation (evident in endpoint logs), and data exfiltration (detectable through user behavior analytics). Each individual phase may appear benign or generate only low-confidence alerts when analyzed in isolation. However, the correlation of weak signals across multiple data sources would reveal the attack campaign's true nature. Current research systems, by focusing on single data modalities, fundamentally cannot detect such distributed attack patterns. Furthermore, the lack of standardized interfaces and data formats across different detection domains creates significant integration barriers. Network intrusion detection systems output alerts in formats incompatible with endpoint detection systems, which in turn use different schemas than user behavior analytics platforms. This heterogeneity forces organizations to deploy multiple independent systems that cannot effectively communicate or correlate findings, creating blind spots that sophisticated adversaries actively exploit. The research community has yet to develop comprehensive frameworks that address both the technical challenges of multi-source integration and the semantic challenges of cross-domain correlation.[5]

Limited Proactive Prediction Capabilities

Most existing systems operate in reactive detection mode, identifying threats only after malicious activities have commenced. True predictive capabilities anticipating attacks before they occur based on precursor indicators and behavioral patterns remain largely unexplored. The distinction between anomaly detection and attack prediction represents a fundamental gap in current research. Traditional intrusion detection systems function as digital alarm systems, triggering alerts when malicious activities are already in progress. This reactive posture provides adversaries with critical windows of opportunity to achieve their objectives before detection occurs. The IBM Cost of a Data Breach Report consistently demonstrates that the time between initial compromise and detection directly correlates with breach severity and financial impact. Organizations that detect breaches within 200 days incur significantly lower costs than those requiring longer detection

periods, yet current detection systems do little to reduce this dwell time through predictive capabilities.

Predictive security requires identifying precursor indicators Subtle behavioral changes, reconnaissance activities, or environmental conditions that precede actual attacks. For instance, an insider planning data exfiltration may exhibit gradual behavioral changes weeks before the actual incident: increased after-hours access, systematic exploration of file systems, or unusual interest in data they don't typically access. Similarly, external attackers conducting reconnaissance generate distinctive patterns in network traffic and system queries before launching actual exploits. Current research has not adequately explored machine learning architectures capable of learning these temporal precursor patterns and generating predictive alerts that enable preemptive defensive actions.

The challenge extends beyond technical detection to include the temporal modeling of attack progression. Attacks unfold over time through distinct phases, yet most detection systems analyze events as independent observations rather than as sequences within temporal attack narratives. Recurrent neural networks and temporal convolutional networks show promise for sequence modeling, but their application to multi-source security telemetry for predictive threat detection remains largely unexplored in academic literature.

Inadequate Handling of Zero-Day Threats

While supervised learning approaches excel at detecting known attack patterns, they fundamentally fail when confronted with zero-day exploits that exhibit no similarity to training data. Conversely, purely unsupervised approaches generate excessive false positives, rendering them impractical for operational deployment. The optimal balance between supervised and unsupervised techniques, particularly in hybrid architectures that leverage the strengths of both paradigms, remains insufficiently explored.

Zero-day vulnerabilities represent one of the most significant challenges in cybersecurity, as they exploit previously unknown software flaws for which no signatures or patches exist. The Stuxnet worm, which exploited four zero-day vulnerabilities simultaneously, demonstrated the devastating potential of such attacks. Traditional signature-based detection systems are fundamentally incapable of detecting zero-day exploits, as they rely on prior knowledge of attack patterns. While supervised machine learning extends detection capabilities beyond exact signature matching by learning generalized patterns from labeled training data, these models still fail when confronted with attack techniques that differ substantially from their training distributions. Unsupervised learning approaches, including autoencoders, isolation forests, and clustering algorithms, offer theoretical solutions by identifying anomalies without requiring labeled attack examples. However, operational deployment of purely unsupervised systems reveals a critical limitation: the false positive problem. In enterprise environments where millions of events occur daily, even a 1% false positive rate generates thousands of false alarms, overwhelming security analysts and rendering the system operationally infeasible. The fundamental challenge lies in distinguishing between benign anomalies (legitimate but unusual activities) and malicious anomalies (actual attacks).

Hybrid architecture that combines supervised and unsupervised learning represents a promising but underexplored research direction. Such systems could leverage supervised models to detect known attack patterns with high precision while employing unsupervised models to identify novel anomalies, with ensemble techniques or meta-learning approaches reconciling their outputs. However, critical questions remain unanswered: How should the outputs of heterogeneous models be weighed and combined? How can hybrid systems adapt as new attack patterns emerge and are incorporated into training data? What architectural patterns enable hybrid systems to maintain both high detection rates and low false positive rates? The research community has yet to develop comprehensive frameworks addressing these questions, particularly for multi-source detection scenarios where different data modalities may benefit from different hybrid architectures.[6]

Insider Threat Detection Challenges

Insider threats present unique challenges due to the legitimate nature of insider credentials and the difficulty of distinguishing malicious intent from normal behavioral variations. Existing user behavior analytics systems struggle with high false positive rates, particularly in dynamic environments where job roles evolve and work patterns change. Adaptive baseline techniques that account for legitimate behavioral evolution while maintaining sensitivity to malicious deviations require further investigation.

The insider threat problem differs fundamentally from external attack detection because insiders possess legitimate credentials, authorized access to sensitive resources, and intimate knowledge of organizational security controls. The 2013 Edward Snowden incident, the 2016 NSA breach by Harold Martin, and numerous corporate espionage cases demonstrate that insider threats can cause catastrophic damage while operating within the bounds of their technical privileges. Traditional perimeter-based security controls offer no protection against insiders who already reside within the perimeter, necessitating behavioral analytics approaches that can identify malicious intent despite legitimate access.

Current user behavior analytics (UBA) systems establish behavioral baselines by learning normal patterns of user activity typical login times, frequently accessed resources, common application usage patterns, and standard data transfer volumes. Deviations from these baselines trigger alerts. However, this approach faces several critical challenges. First, the cold start problem: new employees lack sufficient historical data to establish reliable baselines, yet they may pose elevated risk during their initial employment period. Second, the concept drift problem: legitimate behavioral changes cause baseline deviations that are indistinguishable from malicious activity using purely statistical approaches. Third, the adversarial adaptation problem: sophisticated insiders aware of monitoring systems can gradually shift their behavior over time, causing baselines to adapt to increasingly malicious patterns without triggering alerts.

Furthermore, existing research inadequately addresses the temporal dynamics of insider threats. Malicious insiders rarely execute attacks impulsively; instead, they typically progress through identifiable phases: planning (reconnaissance of systems and data), preparation (establishing exfiltration channels), execution (actual data theft or sabotage), and covering tracks (log manipulation, evidence destruction). Each phase exhibits distinctive behavioral signatures, yet

current systems analyze user activities as independent events rather than as sequences within temporal threat narratives. Machine learning architectures capable of modeling these temporal progressions while adapting to legitimate behavioral evolution represents a significant research gap.

The challenge is further compounded by the extreme class imbalance in insider threat detection. In typical organizational environments, malicious insider incidents occur at rates of perhaps 1 in 10,000 or 1 in 100,000 employees annually, creating datasets where malicious examples are vanishingly rare compared to benign activities. Standard learning approaches struggle with such extreme imbalance, often learning to simply classify all activities as benign to achieve high overall accuracy while completely failing to detect actual threats. Advanced techniques including synthetic minority oversampling, cost-sensitive learning, and anomaly detection specifically designed for extreme imbalance scenarios remain underexplored in the insider threat context.[7]

Scalability and Real-Time Processing Constraints

Enterprise environments generate massive volumes of security telemetry network flows, system logs, and user activity records at rates that challenge real-time processing capabilities. Many sophisticated machine learning approaches, particularly deep learning architectures, impose computational requirements incompatible with real-time operational constraints. Two-stage detection architectures that balance detection accuracy with computational efficiency represent a promising but underexplored approach.

The volume and velocity of security telemetry in modern enterprise environments present formidable computational challenges. A medium-sized organization with 10,000 endpoints may generate hundreds of gigabytes of log data daily, including millions of Windows event log entries, network flow records, and user activity events. Large enterprises with hundreds of thousands of endpoints face data volumes measured in terabytes per day. Processing this data in real-time analyzing events as they occur and generating alerts within seconds or minutes requires computational architectures capable of sustaining throughput rates exceeding millions of events per second.

Deep learning models, while achieving state-of-the-art detection accuracy in research settings, often impose computational requirements that render them impractical for real-time operational deployment. A single forward pass through a deep neural network may require milliseconds to seconds depending on model complexity, which becomes prohibitive when processing millions of events. Furthermore, deep learning models typically require GPU acceleration for acceptable performance, introducing infrastructure costs and deployment complexity that many organizations cannot accommodate. The research community has insufficiently explored the trade-offs between model sophistication and computational efficiency, particularly in the context of multi-source security analytics where multiple models must operate concurrently. Two-stage detection architectures offer a promising approach to managing computational complexity while maintaining detection accuracy. In this paradigm, a lightweight first-stage filter rapidly processes all incoming events, discarding obviously benign activities and forwarding only suspicious events to a more sophisticated second-stage classifier. For example, a simple statistical anomaly detector

or one-class SVM could filter 99.9% of benign events in microseconds per event, while a complex deep learning model analyzes only the remaining 0.1% of suspicious events. This approach reduces overall computational requirements by three orders of magnitude while maintaining high detection accuracy.

However, critical research questions remain unanswered regarding two-stage architectures. How should the first-stage filter be designed to maximize benign event filtering while minimizing false negatives. What is the optimal balance between first stage and second-stage computational budgets? How can two-stage systems adapt over time as attack patterns evolve? How do two-stage architectures perform across different data modalities (network traffic, system logs, user behavior), and should different modalities employ different architectural patterns? The research literature provides limited guidance on these questions, particularly for integrated multi-source detection systems where multiple two-stage pipelines must operate concurrently.

Additionally, the challenge of maintaining model state in real-time systems remains underexplored. Many sophisticated detection approaches require maintaining behavioral baselines, temporal context windows, or model parameters that must be continuously updated as new data arrives. For example, user behavior analytics systems must maintain behavioral profiles for potentially millions of users, each requiring periodic updates as new activities occur. The memory requirements and update latencies associated with maintaining this state at scale present significant engineering challenges that academic research has insufficiently addressed.[8]

Lack of Unified Visualization and Operational Integration

Even when multiple detection systems are deployed, they typically operate as independent silos with separate interfaces, alert formats, and management consoles. Security analysts must manually correlate alerts across systems, a time-consuming and error-prone process. Unified dashboards that aggregate multi-source intelligence, normalize heterogeneous data formats, and provide coherent operational workflows remain rare in both research and commercial offerings.

Modern security operations centers (SOCs) typically deploy multiple specialized detection systems: network intrusion detection systems (NIDS), endpoint detection and response (EDR) platforms, security information and event management (SIEM) systems, user behavior analytics (UBA) tools, and threat intelligence platforms. Each system operates independently with its own management console, alert format, severity classification scheme, and workflow processes. Security analysts must monitor multiple screens, manually correlate alerts across systems, and mentally integrate findings to understand the complete threat landscape. This fragmentation creates several critical operational challenges.

First, the cognitive burden on analysts becomes overwhelming. A typical SOC analyst may need to monitor 5-10 different security tools simultaneously, each generating alerts in different formats with different contextual information. When an alert appears in one system, the analyst must manually query other systems to determine whether related indicators exist elsewhere. This process is time-consuming, error-prone, and fundamentally does not scale as the number of deployed security tools increases. Studies of SOC operations reveal that analysts spend 30-40% of

their time simply gathering and correlating information across disparate systems rather than performing actual threat analysis and response activities.

Second, the lack of standardized alert formats and severity classifications creates confusion and inconsistency. One system may classify an event as "critical" while another rates a similar event as "medium" severity. Alert schemas vary dramatically some systems provide rich contextual information including MITRE ATT&CK mappings and threat intelligence enrichment, while others generate minimal technical alerts requiring extensive manual investigation. This heterogeneity forces analysts to develop system-specific expertise and makes it difficult to establish consistent triage and response procedures across the organization.[9]

Third, the absence of unified authentication and authorization mechanisms creates operational friction. Analysts must maintain separate credentials for each system, navigate different authentication workflows, and manage varying permission models. This not only wastes time but also creates security risks as analysts may resort to insecure practices (password reuse, credential sharing) to manage the complexity. Furthermore, audit trails become fragmented across systems, making it difficult to track analyst activities and ensure accountability.

Research literature has insufficiently addressed these operational integration challenges. While academic papers frequently propose novel detection algorithms and demonstrate their effectiveness on benchmark datasets, they rarely consider how these systems would integrate into existing SOC workflows or interoperate with other security tools. The few research systems that do address integration typically focus on narrow technical aspects without considering the broader operational context including analyst workflows, alert triage procedures, incident response processes and organizational policies. The challenge extends beyond technical integration to include semantic integration ensuring that alerts from different systems can be meaningfully correlated and interpreted within a unified threat narrative. For example, a network intrusion detection system may alert suspicious outbound traffic, while an endpoint detection system simultaneously alerts unusual process behavior, and a user behavior analytics system flags anomalous data access pattern. These three alerts may represent different facets of a single attack campaign, but without semantic integration capabilities, analysts must manually recognize these relationships. Research on automated alert correlation, threat narrative construction, and multi-source evidence fusion remains limited, particularly for systems that integrate more than two data sources.[10]

Limited Integration with Threat Intelligence Frameworks

While the MITRE ATT&CK framework has gained widespread adoption for threat intelligence communication, automated mapping of detection system outputs to ATT&CK techniques remains challenging. Most systems provide low-level technical alerts without contextual information about adversary tactics and techniques, forcing analysts to manually interpret findings within the ATT&CK context.

The MITRE ATT&CK framework has emerged as the de facto standard for describing adversary behaviors, providing a comprehensive taxonomy of tactics and techniques Organizations use ATT&CK to assess defensive coverage, prioritize security investments, communicate threat

intelligence, and structure threat hunting activities. However, a significant gap exists between the low-level technical alerts generated by detection systems and the high-level ATT&CK concepts used for strategic security planning and communication.

Automated ATT&CK mapping presents several technical challenges. First, the mapping is inherently many-to-many: a single technical indicator may correspond to multiple ATT&CK techniques, and a single ATT&CK technique may manifest through numerous technical indicators. Second, accurate mapping often requires contextual information beyond what is available in individual alerts understanding the attack phase, the adversary's apparent objectives, and related activities across multiple systems. Third, ATT&CK techniques are described at varying levels of abstraction, with some techniques having multiple sub-techniques, creating ambiguity about the appropriate mapping granularity.

Current research has explored rule-based approaches to ATT&CK mapping, where domain experts manually define mappings between detection signatures and ATT&CK techniques. However, these approaches suffer from several limitations. They require extensive manual effort to create and maintain mappings as both detection systems and the ATT&CK framework evolve. They cannot handle novel attack variations that don't match predefined rules. They struggle with context-dependent mappings where the same technical indicator maps to different techniques depending on surrounding circumstances. Machine learning approaches to automated ATT&CK mapping remain largely unexplored, particularly for multi-source detection systems where mapping decisions should consider evidence from multiple data sources.

Furthermore, the research community has insufficiently explored how ATT&CK integration can enhance detection capabilities beyond simply labeling alerts. For example, ATT&CK-aware detection systems could leverage knowledge of typical attack sequences (tactics typically progress from initial access through execution, persistence, privilege escalation, defense evasion, credential access, discovery, lateral movement, collection, and exfiltration) to improve threat prediction and prioritization. Systems could identify incomplete attack chains and predict likely next steps, enabling proactive defensive measures. They could assess defensive coverage by identifying ATT&CK techniques for which detection capabilities are weak or absent. However, these advanced applications of ATT&CK integration remain largely theoretical, with limited practical implementations or empirical evaluations in research literature.[11]

Insufficient Evaluation on Realistic, Diverse Datasets

Many research systems are evaluated on outdated or synthetic datasets that fail to capture the complexity and diversity of modern attack techniques. The lack of standardized, realistic evaluation benchmarks spanning multiple data modalities (system calls, user behavior, network traffic, endpoint logs) hinders comparative assessment of integrated detection approaches.

The evaluation of cybersecurity detection systems presents unique challenges that distinguish it from other machine learning application domains. Unlike image classification or natural language processing, where large, diverse, publicly available datasets enable rigorous comparative evaluation, cybersecurity research suffers from a scarcity of realistic, representative datasets.

Organizations understandably hesitate to share security telemetry due to privacy concerns, competitive sensitivity, and potential exposure of vulnerabilities. Consequently, researchers often resort to outdated public datasets, synthetic data generation, or small-scale laboratory experiments that inadequately represent real-world operational environments.

The widely used KDD Cup 1999 and NSF-KDD datasets, despite being over two decades old, continue to appear in contemporary research papers. These datasets reflect network attack techniques from the late 1990s and early 2000s—a technological era predating modern threats such as advanced persistent threats, ransomware, supply chain attacks, and living-off-the-land techniques. Detection systems optimized for these outdated datasets may perform poorly against contemporary attack methods. Furthermore, these datasets focus exclusively on network traffic, providing no information about system calls, user behavior, or endpoint logs, making them unsuitable for evaluating integrated multi-source detection approaches. More recent datasets such as CIC-IDS2017, CSE-CIC-IDS2018, and UNSW-NB15 represent improvements in terms of attack diversity and realism, but they still exhibit significant limitations. First, they remain network-centric, lacking the multi-modal data (system calls, user behavior, endpoint logs) necessary for evaluating integrated detection systems. Second, they are generated in controlled laboratory environments using simulated attack scenarios, which may not accurately reflect the complexity, noise, and ambiguity of real-world enterprise networks. Third, they typically contain balanced or semi-balanced class distributions, whereas real-world environments exhibit extreme class imbalance with attack traffic representing perhaps 0.01-0.1% of total traffic. Detection systems evaluated on balanced datasets may perform poorly in operational deployment where false positive rates become the dominant concern.

The insider threat domain faces even more severe dataset limitations. The CERT Insider Threat Dataset, while valuable, consists of synthetic data generated through simulation rather than actual insider incidents. The scenarios, while informed by real-world case studies, may not capture the full complexity and diversity of insider threat behaviors. Furthermore, the dataset's relatively small size pales in comparison to enterprise environments with tens of thousands of users generating years of activity data. The extreme rarity of actual insider incidents in the dataset (dozens of incidents among thousands of users) reflects reality but creates severe class imbalance challenges that many research papers inadequately address.[12]

System call datasets face similar challenges. The AWSCTD provides valuable Windows system call traces for both benign and malicious applications, but it focuses primarily on malware detection rather than the broader spectrum of attack techniques including living-off-the-land attacks that leverage legitimate system tools. The dataset’s-controlled generation environment may not capture the variability and noise present in real-world endpoint telemetry.

Beyond individual data set limitations, the research community lacks standardized evaluation protocols for integrated multi-source detection systems. How should systems be evaluated when they integrate network traffic, system calls, user behavior, and endpoint logs? Should evaluation use separate datasets for each modality, or should it employ integrated datasets that capture all modalities simultaneously? How should temporal aspects be evaluated, can systems be tested on static datasets or do they require streaming evaluation that captures temporal dependencies and

concept drift? What metrics appropriately capture the trade-offs between detection accuracy, false positive rates, computational efficiency, and operational usability? The research literature provides limited guidance on these questions, making it difficult to compare different approaches or assess their readiness for operational deployment.

Furthermore, most research evaluations focus narrowly on detection accuracy metrics while neglecting other critical operational considerations. How quickly can systems process incoming data? What computational resources do they require? How do they handle missing or corrupted data? How do they adapt to evolving attack techniques? How usable are there interfaces for security analysts? These questions remain largely unaddressed in academic evaluations, creating a gap between research prototypes and operationally deployable systems.

1.3 Research Problem

The central research problem addressed by this investigation can be articulated as follows:

How can an integrated, multi-layered machine learning framework effectively predict both known and zero-day cyber-attacks in real-time by correlating insights from system call patterns, user behavioral analytics, endpoint log sequences, and network traffic analysis, while maintaining operational feasibility through manageable false positive rates and computational efficiency.

This overarching problem encompasses several specific sub-problems:

Multi-Source Data Integration Challenge

Problem Statement: Security telemetry from different sources (system calls, user behavior, network traffic, endpoint logs) exhibits heterogeneous formats, temporal granularities, and semantic representations. How can these diverse data streams be effectively normalized, temporally aligned, and integrated to enable coherent cross-layer threat detection.

The fundamental challenge of multi-source integration extends beyond simple data format conversion to encompass deeper semantic and temporal alignment issues. System calls are captured at microsecond granularity as discrete kernel-level events, network traffic is aggregated into flows spanning seconds to minutes, user behavior is typically analyzed at hourly or daily granularity, and endpoint logs arrive as asynchronous events with varying timestamp precision. Integrating these data streams requires not only converting formats but also establishing temporal correspondence determining which system calls, network flows, user activities, and endpoint log entries relate to the same underlying security event or attack campaign.

Furthermore, the semantic gap between low-level technical indicators and high-level security concepts presents significant challenges. A system call sequence like "**CreateProcess → WriteFile → CreateRemoteThread**" has clear technical meaning but requires contextual interpretation to determine whether it represents legitimate application behavior, benign system administration, or malicious code injection. Similarly, network traffic showing repeated connection attempts to multiple IP addresses could indicate legitimate service discovery,

misconfigured applications, or malicious port scanning. Bridging this semantic gap requires not only analyzing individual data sources but also correlating evidence across sources to construct coherent threat narratives.

Specific Challenges:

● Data format heterogeneity: Binary system call traces, textual log entries, numerical network flow features, and structured user activity records require different parsing, preprocessing, and feature extraction approaches.

● Temporal misalignment: Different data sources operate at different time scales (microseconds for system calls, seconds for network flows, hours for user behavior), requiring sophisticated temporal alignment and aggregation strategies.

● Semantic gap: Low-level technical features (specific system calls, packet headers, log event IDs) must be mapped to high-level security concepts (attack techniques, adversary tactics, threat indicators) to enable meaningful analysis.

● Volume and velocity constraints: Real-time processing requires handling millions of events per second across multiple data streams while maintaining temporal coherence and enabling cross-source correlation.

● Missing and incomplete data: Real-world environments exhibit data loss, sensor failures, and incomplete telemetry coverage, requiring robust handling of missing data and graceful degradation when some data sources are unavailable.[13]

Hybrid Learning Architecture Design

Problem Statement: Supervised learning excels at detecting known attacks but fails on zero-day threats, while unsupervised learning detects anomalies but generates excessive false positives. How can hybrid architecture optimally combine supervised and unsupervised techniques to achieve both high detection rates for known threats and effective identification of novel attack patterns?

The tension between supervised and unsupervised learning represents a fundamental challenge in cybersecurity machine learning. Supervised approaches (Random Forest, XGBoost, neural networks) learn from labeled examples of attacks and benign activities, achieving high accuracy on attack types like their training data. However, they exhibit poor generalization to novel attack techniques that differ substantially from training examples precisely the zero-day threats that pose the greatest risk. Unsupervised approaches identify anomalies without requiring labeled attack examples, theoretically enabling zero-day detection. However, they cannot distinguish between benign anomalies and malicious anomalies, resulting in false positive rates that render them operationally infeasible. Hybrid architecture that combines both paradigms offer theoretical advantages but introduces complex design questions. Should supervised and unsupervised models operate in parallel, with their outputs combined through ensemble methods? Should they operate sequentially, with unsupervised models filtering data before supervised classification? Should they

operate hierarchically, with different models analyzing different data sources or different aspects of the same data? How should model outputs be weighed and combined through simple voting, weighted averaging, stacking with meta-learners or more sophisticated fusion approaches?

Specific Challenges:

● Optimal model selection: Different data modalities (system calls, user behavior, network traffic, endpoint logs) may benefit from different machine learning approaches. How should models be selected for each modality to maximize detection effectiveness?

● Effective ensemble strategies: When combining heterogeneous models (supervised and unsupervised, different algorithms, different data sources), how should their outputs be reconciled to produce coherent final predictions?

● Balancing detection sensitivity with false positive rates: Hybrid systems must maintain high detection rates for both known and novel attacks while keeping false positive rates below operational thresholds.

● Handling class imbalance: Security datasets exhibit extreme class imbalance (attacks represent 0.01-0.1% of events), requiring specialized techniques beyond standard machine learning approaches.

● Model adaptation and evolution: As new attack patterns emerge and are incorporated into training data, how should hybrid systems adapt their supervised components while maintaining unsupervised capabilities for detecting future novel attacks?

Behavioral Baseline Establishment and Adaptation

Problem Statement: User behavior and system activity patterns evolve over time due to legitimate factors (role changes, organizational restructuring, technology adoption). How can detection systems establish behavioral baselines that adapt to legitimate changes while maintaining sensitivity to malicious deviations?

Behavioral baseline establishment faces the fundamental challenge of distinguishing between legitimate behavioral evolution and malicious drift. When a user's file access patterns change dramatically, does this reflect a new project assignment, a role transition, or malicious data reconnaissance? When system call patterns shift, does this indicate software updates, configuration changes, or malware infection? Traditional static baseline approaches fail in dynamic environments, generating excessive false positives as legitimate behaviors evolve. However, overly adaptive baselines risk incorporating malicious behaviors, causing systems to gradually accept increasingly suspicious activities as "normal." The challenge is compounded by the cold start problem new users, new systems, and new applications lack historical data for baseline establishment, yet they may pose elevated security risks during their initial operational period. Furthermore, different behavioral aspects may evolve at different rates: login times may change gradually over weeks, while file access patterns may shift abruptly with new project assignments.

Effective baseline systems must accommodate these varying temporal dynamics while maintaining detection sensitivity.

Specific Challenges:

● Distinguishing legitimate behavioral evolution from malicious drift: Developing techniques that can differentiate between benign behavioral changes and malicious activities (reconnaissance, data exfiltration preparation) without requiring manual analyst review of every deviation.

● Handling concept drift in dynamic environments: Adapting baselines to accommodate legitimate changes in user roles, organizational structure, technology adoption, and business processes while maintaining sensitivity to attacks.

● Establishing baselines with limited historical data: Addressing the cold start problem for new users, new systems, and new applications that lack sufficient historical data for reliable baseline establishment.

● Managing computational overhead of continuous model updates: Maintaining behavioral baselines for potentially millions of users and systems requires efficient update mechanisms that can operate in real-time without excessive computational costs.

● Temporal modeling of behavioral evolution: Capturing the temporal dynamics of how behaviors legitimately evolve over different time scales while detecting anomalous temporal patterns indicative of attacks.

Real-Time Processing and Scalability

Problem Statement: Enterprise environments generate security telemetry at rates exceeding millions of events per second. How can sophisticated machine learning models operate within real-time latency constraints while maintaining detection accuracy?

The scalability challenge encompasses both throughput (events processed per second) and latency (time from event occurrence to alert generation). Real-time threat detection requires processing events as they occur, generating alerts within seconds to minutes to enable timely response. However, sophisticated machine learning models particularly deep learning architectures, may require milliseconds to seconds per prediction, creating throughput bottlenecks when processing millions of events. Furthermore, maintaining behavioral state (user baselines, temporal context windows, model parameters) for large-scale environments introduces memory and update latency challenges.

The problem is exacerbated in multi-source detection scenarios where multiple models must operate concurrently on different data streams. A system integrating network traffic analysis, system call monitoring, user behavior analytics, and endpoint log inspection may need to run 10-20 different machine learning models simultaneously, each processing its respective data stream in real-time. The aggregate computational requirements may exceed available resources, forcing difficult trade-offs between detection sophistication and operational feasibility.[14]

Specific Challenges:

● Computational complexity of deep learning models: Deep neural networks, while achieving high accuracy, impose computational requirements (GPU acceleration, high memory usage, long inference times) that may be incompatible with real-time processing of high-volume data streams.

● Memory requirements for maintaining behavioral state: Behavioral analytics systems must maintain state for potentially millions of users and systems, requiring efficient data structures and update mechanisms that can operate within memory constraints.

● Latency constraints for real-time alerting: Security operations require alerts within seconds to minutes of attack occurrence, constraining the computational budget available for each detection decision.

● Scalability to enterprise-scale data volumes: Systems must scale from laboratory environments to enterprise environments without proportional increases in computational resources.

● Resource allocation across multiple detection components: In integrated multi-source systems, how should computational resources be allocated across different detection components to maximize overall detection effectiveness within resource constraints?

Operational Integration and Usability

Problem Statement: Security analysts require coherent, actionable intelligence rather than fragmented technical alerts. How can multi-source detection systems present unified, contextualized findings that enable efficient threat triage and response. The operational integration challenge extends beyond technical system integration to encompass human factors, workflow integration, and organizational processes. Security analysts are not machine learning experts; they are domain specialists who need to understand what attacks are occurring, what systems are affected, what actions should be taken, and what priority should be assigned to different threats. Raw machine learning outputs provide insufficient context for operational decision-making.

Furthermore, the alert fatigue problem where analysts become overwhelmed by excessive alerts and begin ignoring or dismissing them represents a critical operational challenge. Even with false positive rates of 5%, a system processing millions of events daily may generate thousands of false alarms, overwhelming analyst capacity. Effective operational integration requires not only accurate detection but also intelligent alert prioritization, contextual enrichment, and workflow integration that enables analysts to efficiently triage and respond to threats.

Specific Challenges:

● Alert normalization across heterogeneous detection engines: Different detection components generate alerts in different formats with different severity classifications, confidence metrics, and contextual information. How can these be normalized into a consistent schema that enables coherent analysis?

● Contextual enrichment with threat intelligence: Raw technical alerts require enrichment with contextual information including MITRE ATT&CK mappings, threat intelligence indicators, affected asset criticality, and related alerts to enable informed analyst decision-making.

● Prioritization and ranking of alerts by severity and confidence: With potentially thousands of alerts daily, how should systems prioritize which alerts require immediate attention versus which can be deferred or automated?

● Workflow integration with existing security operations processes: Detection systems must integrate with existing incident response workflows, ticketing systems, SIEM platforms, and organizational procedures rather than requiring analysts to adopt entirely new processes.

● Usability and interpretability for non-expert users: Security analysts need to understand why alerts were generated, what evidence supports them, and what actions are recommended, requiring interpretable machine learning approaches and intuitive user interfaces.

2. Methodology

This project's methodology aimed to create an integrated Real Time Cyber Threats Predicting System that predicts both known and zero-day cyber threats by analyzing multiple security data sources. Rather than relying on a single detection method, the overall approach uses multiple components driven by machine learning, with each component focusing on a different perspective on threats. The project integrates analysis of system call traces, user behavior, network traffic patterns and Windows endpoint logs to form a more proactive cybersecurity framework. This multi-layered design is crucial because cyberattacks can manifest in various ways. Some attacks show up in process behavior, others in user activity, some in endpoint telemetry and others in unusual network traffic. Therefore, the methodology views each of these as a separate yet complementary source of threat intelligence.

At the methodological level, the project is divided into four research and implementation streams. The system call component employs a hybrid detection pipeline that turns raw process activity into structured traces, transforms them into both aggregate and sequential representations, and then analyzes them using a Random Forest gatekeeper followed by an RNN fallback for uncertain cases. The user behavior component uses an LSTM-based UEBA model to capture user activity patterns over a 14-day window with engineered behavioral features. The network traffic component applies to a hybrid anomaly detection method that combines an Autoencoder, Isolation Forest, and a logistic regression meta-classifier to find possible zero-day traffic anomalies. The endpoint log component analyzes Sysmon and PowerShell logs using a structured feature engineering pipeline and a multi-class XGBoost model to classify known attack categories such as credential dumping, defense evasion, lateral movement, and privilege escalation. Together, these choices allow the entire system to support both anomaly-based and classification-based predictions in real time.

The methodology also follows a common pattern across components: data collection, preprocessing, feature engineering, model training, prediction, and alert-oriented output

integration. While each component uses different data types and machine learning models, they all share this design logic. First, raw cybersecurity data is gathered from its respective source, then normalized into machine-readable structures, transformed into relevant feature spaces and finally used by predictive models to produce threat scores, labels or anomaly decisions. This method maintains consistency across the project and supports future integration into a unified platform.[15]

2.1 Introduction to Methodology

The methodology in this project is based on the idea that effective cyber threat prediction requires smart analysis of behavior rather than just relying on predefined signatures. Traditional signature-based systems work well only when known attack patterns already exist, but they struggle with evolving attacks, stealthy insider activity and zero-day threats. For this reason, the proposed project employs a data-driven methodology that utilizes machine learning models to recognize patterns of normal and malicious behavior from various security data sources. This makes the project suited for predicting known attacks and identifying zero-day threats.

This methodology is intentionally hybrid instead of relying on a single model. In the network traffic component, the project combines the strengths of the Autoencoder and Isolation Forest while using logistic regression as a meta-classifier to make final decisions. In the system call component, the methodology uses a fast first-stage Random Forest for obvious cases, sending only ambiguous traces to the RNN, which enhances efficiency while allowing deeper sequential analysis when necessary. In the endpoint log component, it combines TF-IDF textual encoding with numerical behavioral indicators, enabling the model to capture both semantic and structured evidence from logs. In the user behavior component, it employs temporal sequence modeling to detect slow or stealthy behavior changes more effectively than static models. These choices demonstrate that the methodology is not only predictive but also structured for practical operational use.

Another key principle of this methodology is context preservation. The system does not just count isolated events; it strives to maintain the relationships among actions, timestamps, sessions, event sequences, process context and command-line content. For instance, endpoint logs are grouped into sessions to model a chain of actions rather than as disconnected records. System call traces preserve API ordering for sequential learning. User behavior is modeled over a time window rather than at a single event point and network traffic is evaluated through deviations from learned normal patterns. This design is meant to detect behavior as a process rather than just isolated incidents.

2.2 Requirement Gathering

Requirement gathering for this project involves figuring out what the proposed system needs to do, the type of data it must manage, the prediction capabilities it must provide and how it should

operate in real cybersecurity environments. According to the four component documents, the project requires the ability to ingest and analyze various security data sources in near real time, including Windows endpoint logs, system calls, user activity logs and network traffic. This requirement arose from the project's goal to predict both known and zero-day attacks instead of focusing on a single narrow attack type. The requirement gathering process also shows that the system must support multiple machine learning workflows. Some threats need supervised classification, like predicting known attacks from endpoint logs, while others need unsupervised or anomaly-based detection, such as predicting zero-day traffic. Additionally, the system must support sequence-based modeling for user behavior and system calls because solely analyzing static features cannot capture temporal or sequential patterns. Therefore, the project requirements extend beyond a single model type; they include support for deep learning, ensemble learning, anomaly detection, feature engineering, threshold-based decisions and model integration.

From an operational standpoint, the requirements also indicate that the final system should facilitate real-time or near-real-time security workflows. The endpoint component directly aims for real-time analyst alerts and dashboard integration. The user behavior component considers latency, throughput and uptime expectations, indicating that the design is not merely theoretical but meant for real-world use. The system call component includes monitoring endpoints, live process tracking, and streaming support, showing that real-time ingestion and analysis are crucial. These factors suggest that requirement gathering considered not only prediction accuracy but also response speed, scalability and usability in analyst workflows.

Another major requirement identified from the component designs is the need for feature-rich preprocessing pipelines. The project requires normalized schemas, session construction, TF-IDF vectorization, behavioral indicators, sequence encoding, numerical scaling and threshold-based classification logic. These are necessary because raw cybersecurity data tends to be noisy and semi-structured, making it hard to use directly in machine learning models. Thus, requirement gathering in this project naturally included both data-level needs and modeling requirements.

2.3 Stakeholder Analysis

Several key stakeholders can be identified for this project. The primary group is security analysts and SOC personnel. The system is designed to assist these analysts by providing alerting, threat triage, session review, and dashboard visualization. The endpoint log component highlights that the expected results will feed into analyst workflows for attack labeling, session review and threat investigation. This indicates that the methodology was developed with operational cybersecurity teams in mind, rather than just for academic use. The second important stakeholder group is organizations and enterprise security administrators. These stakeholders gain from the project since they need proactive visibility into known and unknown cyber threats across their infrastructure. The endpoint component focuses on Windows enterprise telemetry, the user behavior component examines insider threats and unusual activity in enterprise systems, the network traffic component detects zero-day anomalies in network communication, and the system

call component supports process-level malware analysis. This shows the project's relevance for enterprise environments, where no single data source is enough for complete threat visibility.

The third stakeholder group includes incident response and digital forensics teams. Since the project maintains session context, process chains, command-line semantics and sequential behavior, its outputs are valuable for initial detection and for later investigation and evidence review. For example, endpoint telemetry includes process names, command lines, registry paths, event chains and suspicious execution indicators. System call analysis preserves structured trace content and process-level behavior. These outputs help responders understand why a threat was predicted and the activity chain that led to that decision.

The fourth stakeholder group is system owners and deployment engineers who need the system to work reliably, scale properly and connect with monitoring infrastructure. The user behavior component offers measurable operational expectations such as inference latency, throughput and uptime. The system call component includes monitor endpoints and live ingestion mechanisms. The endpoint component is designed as a modular real-time pipeline. These aspects show that deployment-oriented stakeholders are crucial since the project aims to advance beyond isolated model testing to practical integration.

Finally, researchers and academic evaluators are also stakeholders. The project is built as a research-driven integrated framework. It adds value by combining various cybersecurity data sources and different machine learning strategies into a broader real-time predictive architecture. This makes the work relevant for future research in hybrid cyber threat intelligence and predictive security analytics.

2.4 System Architecture Overview

The system architecture of the project features a multi-layered predictive cybersecurity framework. Four specialized components work on different data streams and contribute to a wider real-time defense system. At a high level, the architecture starts with data acquisition from multiple sources, proceeds through preprocessing and feature engineering pipelines, applies machine learning models for prediction or anomaly detection and routes the results to analyst-facing outputs like alerts, labels or dashboards. This layered approach reflects how cyber threats show up differently at the host, user, process and network levels.

Within this architecture, the system call component acts as a process-level behavioral analysis layer. It captures Sysmon events, converts them into CAPEv2-style call records, creates both Random Forest features and RNN sequences and applies a two-stage inference process. The Random Forest manages clear cases while the RNN examines uncertain traces. This enhances both speed and sequence awareness.

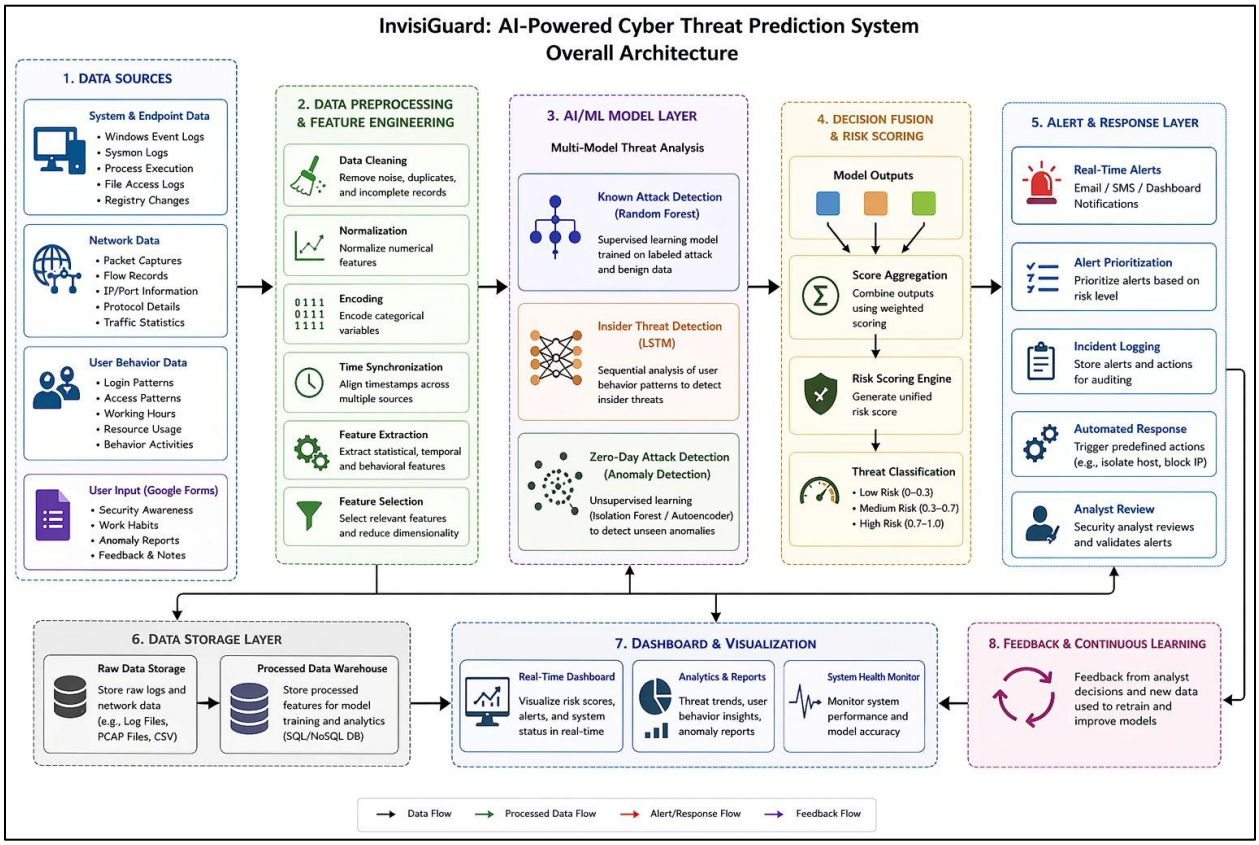
The user behavior component serves as a user-centric analytics layer. It models normal and abnormal user behavior over time using a 14-day sequence window and 24 behavioral features derived from file activity, USB usage, temporal behavior and cross-signal indicators. This allows

architecture to detect insider-style anomalies and gradual misuse patterns that may not be seen from network or host telemetry alone.

The network traffic component functions as a zero-day anomaly detection layer. It learns normal traffic patterns and identifies deviations using a hybrid framework that combines Autoencoder reconstruction error and Isolation Forest anomaly scoring. It then uses logistic regression to make a final threat decision. This aspect of the architecture is particularly valuable for spotting unknown threats that do not conform to known attack signatures.

The endpoint log component acts as a host telemetry intelligence layer. It processes Sysmon and PowerShell data, standardizes heterogeneous records, groups related events into sessions, generates 369-dimensional features and uses XGBoost to classify known attack categories. It is also designed for real-time workflow integration by forwarding predicted results to alerting and dashboard layers.[16]

Together, architecture can be viewed as a pipeline with these logical stages:



The architecture is modular, allowing each component to operate independently while still contributing to a broader integrated system. This modularity is essential since it enables future system-level integration without forcing all data types into a single model.

2.5 Functional Requirements

1. Multi-source data collection

The system must collect data from four different cybersecurity sources: system call traces, user behavior logs, network traffic, and Windows endpoint logs. This is a key requirement because the project is designed as an integrated cyber threat prediction system, not just a single-source detector. The system call component relies on process activity traces, the user behavior component depends on user activity sessions, the network component uses traffic records, and the endpoint component works with Sysmon and PowerShell logs.

2. Data preprocessing and normalization

The system must preprocess raw security data and convert it into machine-readable formats before model training or prediction. This involves normalizing traces in the system call component, constructing behavioral sessions in the user behavior component, preparing traffic in the network component, and unifying schema plus grouping sessions in the endpoint log component. Without preprocessing, raw data would be too noisy and inconsistent for effective machine learning analysis.

3. Feature extraction for each component

The system must generate relevant features for each subcomponent based on the nature of its data. For the system call component, this includes Random Forest features and RNN sequences. For the user behavior component, it includes 24 engineered behavioral features. For the network traffic component, it includes reconstruction error and anomaly scores. For the endpoint component, it includes TF-IDF features and 19 numerical behavioral indicators. This requirement is necessary because each model relies on a specific representation of the original security data.

4. Known attack prediction

The system must predict known cyberattacks using relevant data sources. This is primarily needed in the system call component, user behavior component, and endpoint log component. The endpoint model must classify known attacks like credential dumping, defense evasion, lateral movement, and privilege escalation. The user behavior model must identify malicious or suspicious user actions. The system call component must determine if behavior is benign or related to malware.

5. Unseen attack prediction

The system must identify zero-day or previously unseen threats, especially from network traffic patterns. This is a major requirement for the network traffic component, where the system learns

normal traffic behavior and flags anomalies that may suggest unknown attacks. This requirement is vital because the main research goal includes predicting both known and zero-day attacks.

6. Temporal and sequential behavior analysis

The system must analyze patterns over time, not just isolated events. This is important because cyber threats often occur as sequences of actions instead of single suspicious events. The user behavior component uses a 14-day LSTM sequence window, the system call component uses RNN sequence analysis, and the endpoint component groups events into sessions and event chains. This allows the system to identify gradual, stealthy, and context-dependent attacks.

7. Hybrid model-based decision making

The system must combine multiple detection methods when necessary to improve prediction quality. In the network traffic component, Autoencoder and Isolation Forest outputs are combined through logistic regression. In the system call component, Random Forest acts as a first-stage gatekeeper, passing uncertain cases to RNN. This requirement supports stronger and more reliable predictions than relying on just one model.

8. Threat classification and anomaly labeling

The system must provide final predictions in the form of labels, classes, or anomaly decisions. For example, the endpoint component must output one of five classes, the user behavior component must give anomaly probability and a normal/threat decision, the network component must indicate anomaly or normal status, and the system call component must provide benign, malware, or grey-zone routing results. This is necessary to make predictions practical.

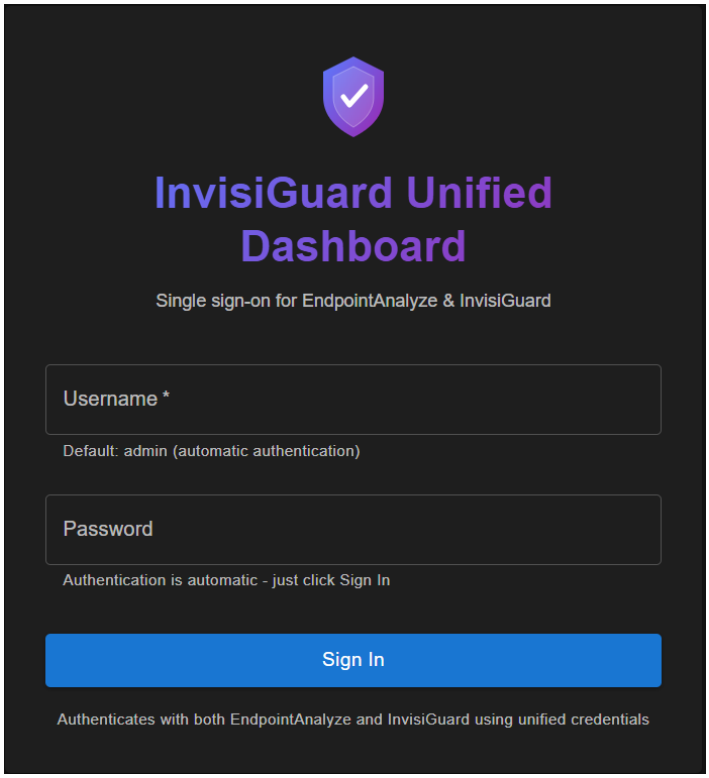
9. Real-time or near-real-time monitoring support

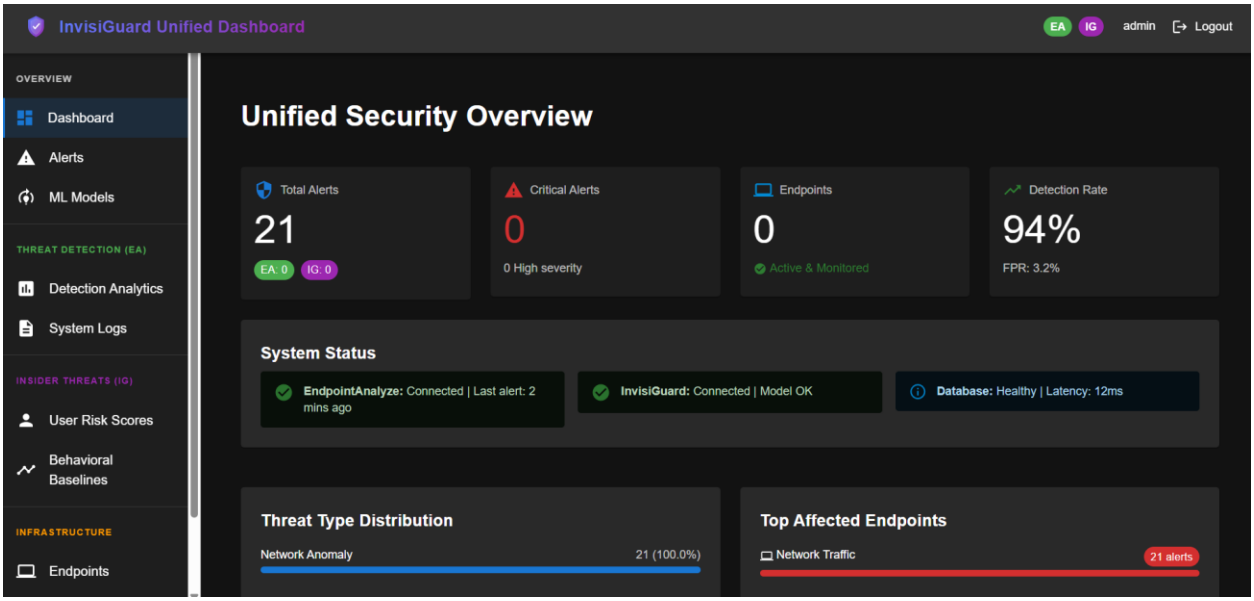
The system must support real-time or near-real-time cybersecurity monitoring. The system call component includes live monitoring endpoints and streaming, the endpoint component is designed as a near-real-time pipeline, and the user behavior component includes latency and throughput expectations. This requirement directly relates to the goal of building a Real Time Cyber Threats Predicting System.

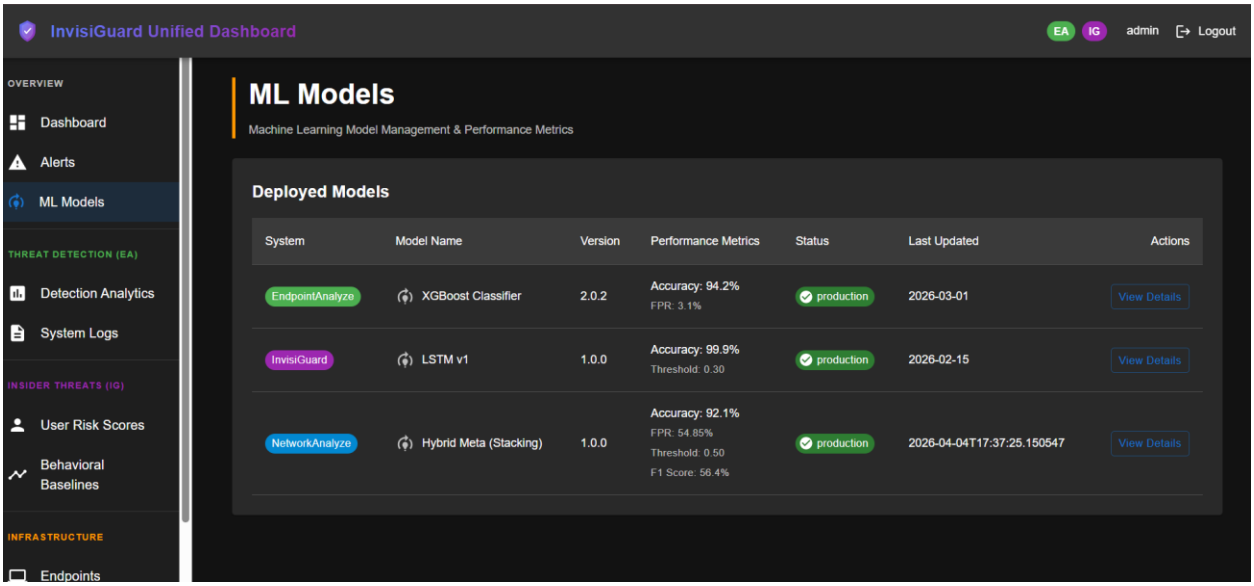
10. Alerting and analyst support

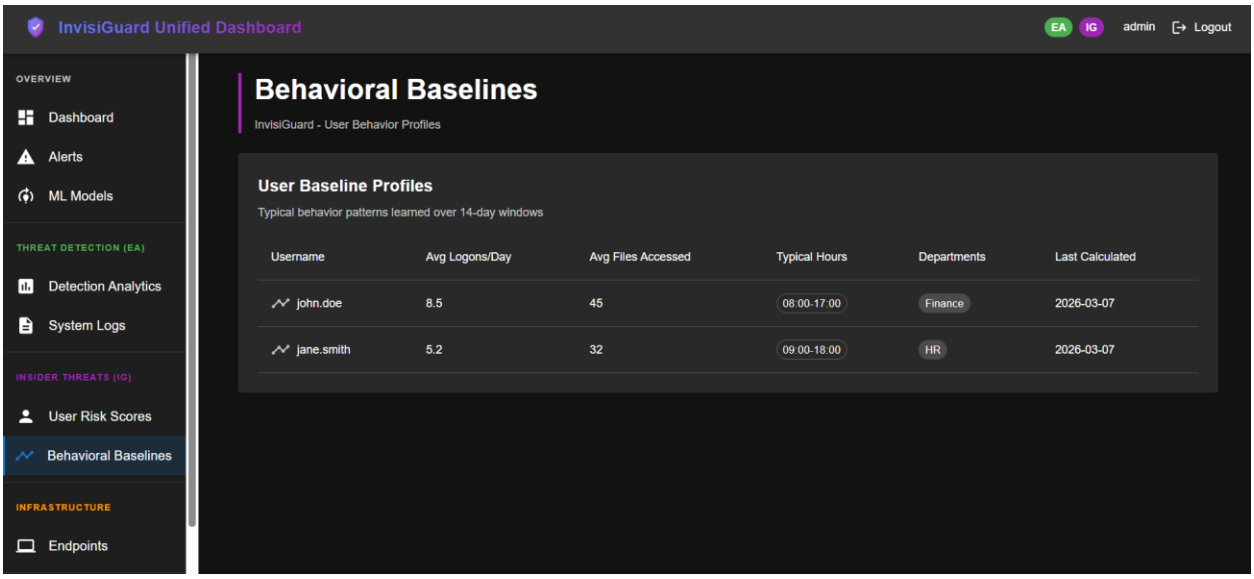
The system must provide outputs that assist analysts in decision-making, such as attack labels, threat alerts, session context, and dashboard-ready results. This is especially evident in the endpoint and system call components, where outputs are meant for alerting, session review, and analyst workflows. The system is expected not only to detect threats but also to present useful threat intelligence.

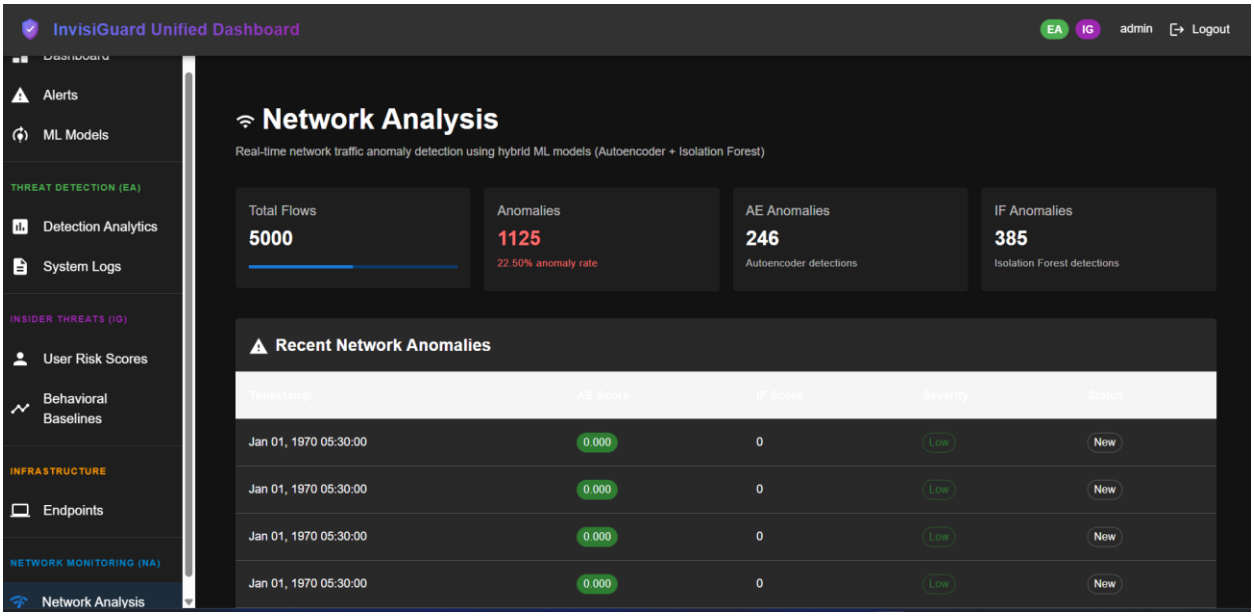
Front-end


InvisiGuard Unified Dashboard
Single sign-on for EndpointAnalyze & InvisiGuard
Username *
Default: admin (automatic authentication)
Password
Authentication is automatic - just click Sign In
Sign In
Authenticates with both EndpointAnalyze and InvisiGuard using unified credentials


InvisiGuard Unified Dashboard
EA
IG
admin
Logout
OVERVIEW
Dashboard
Alerts
ML Models
THREAT DETECTION (EA)
Detection Analytics
System Logs
INSIDER THREATS (IG)
User Risk Scores
Behavioral Baselines
INFRASTRUCTURE
Endpoints
Unified Security Overview
Total Alerts
21
EA: 0
IG: 0
Critical Alerts
0
0 High severity
Endpoints
0
Active & Monitored
Detection Rate
94%
FPR: 3.2%
System Status
EndpointAnalyze: Connected | Last alert: 2 mins ago
InvisiGuard: Connected | Model OK
Database: Healthy | Latency: 12ms
Threat Type Distribution
Network Anomaly
21 (100.0%)
Top Affected Endpoints
Network Traffic
21 alerts


InvisiGuard Unified Dashboard
EA
IG
admin
Logout
OVERVIEW
Dashboard
Alerts
ML Models
THREAT DETECTION (EA)
Detection Analytics
System Logs
INSIDER THREATS (IG)
User Risk Scores
Behavioral Baselines
INFRASTRUCTURE
Endpoints
ML Models
Machine Learning Model Management & Performance Metrics
Deployed Models
System
Model Name
Version
Performance Metrics
Status
Last Updated
Actions
EndpointAnalyze
XGBoost Classifier
2.0.2
Accuracy: 94.2%
FPR: 3.1%
production
2026-03-01
View Details
InvisiGuard
LSTM v1
1.0.0
Accuracy: 99.9%
Threshold: 0.30
production
2026-02-15
View Details
NetworkAnalyze
Hybrid Meta (Stacking)
1.0.0
Accuracy: 92.1%
FPR: 54.85%
Threshold: 0.50
F1 Score: 56.4%
production
2026-04-04T17:37:25.150547
View Details


InvisiGuard Unified Dashboard
EA
IG
admin
Logout
OVERVIEW
Dashboard
Alerts
ML Models
THREAT DETECTION (EA)
Detection Analytics
System Logs
INSIDER THREATS (IG)
User Risk Scores
Behavioral Baselines
INFRASTRUCTURE
Endpoints
Behavioral Baselines
InvisiGuard - User Behavior Profiles
User Baseline Profiles
Typical behavior patterns learned over 14-day windows
Username
Avg Logons/Day
Avg Files Accessed
Typical Hours
Departments
Last Calculated
john.doe
8.5
45
08:00-17:00
Finance
2026-03-07
jane.smith
5.2
32
09:00-18:00
HR
2026-03-07



2.6 Non-Functional Requirements

1. Accuracy

The system must maintain high predictive accuracy across all four components, ensuring that its outputs are reliable for cybersecurity operations. This goal matches the reported model performances: the network hybrid model achieved 92.12% accuracy, the user behavior model reached 94.3% accuracy, and the endpoint model obtained 90.28% accuracy. For the system call component, the design aims to improve output quality by combining a Random Forest with an RNN fallback, rather than relying on just one method. High accuracy is crucial since incorrect predictions can undermine trust in the entire platform.

2. Low false positive behavior

The system should reduce false alarms to prevent analysts from being overwhelmed by unnecessary alerts. This is particularly important for the network traffic, user behavior, and endpoint components, where anomaly detection and multiclass classification can generate misleading warnings. The user behavior component emphasizes high precision and low false positives, while the network traffic component improves upon the limitations of single-model false positives through a hybrid approach. This requirement is vital for practical analyst adoption.

3. Performance efficiency

The system must perform predictions quickly enough for operational use. This requirement applies to all four components. The user behavior model reports under 200 ms for machine learning inference and less than 1 second for end-to-end latency. The system call component uses the Random Forest as a fast first-stage gatekeeper, allowing the RNN to run only when necessary. The endpoint component is built for nearly real-time behavioral analysis. The network component

employs a hybrid architecture that supports real-time anomaly scoring. The project calls for efficient processing, not just correct outcomes.

4. Scalability

The system should handle large volumes of security events, sessions, traces, and traffic records. This requirement aligns strongly with the user behavior component, which manages 10,000 events per second and 500 predictions per second, as well as the endpoint component, which discusses broader architectural compatibility with upstream anomaly filtering for scalability. It also connects with the multi-source nature of the entire project, since real enterprise deployment would generate significant amounts of logs and traffic from all four data sources.

5. Reliability

The system must operate consistently, providing dependable outputs during continuous monitoring. This requirement corresponds to the user behavior component's expectation of 99.9% uptime, as well as the system call component's monitoring capabilities, health checks and management of failure cases, such as no loaded models or improperly formatted traces. Reliability is crucial, as threat prediction becomes less valuable if the system is unstable or unavailable during active monitoring.

6. Real-time responsiveness

The system must respond quickly enough to enable proactive cybersecurity defense. This differs from general efficiency because it emphasizes timeliness from when data arrives to when predictions are output. The user behavior component sets specific timing targets, the system call component includes live stream and monitoring endpoints, and the endpoint component describes a near-real-time prediction workflow from event collection to alerting. The network traffic component also aims to highlight anomalous patterns in real time. This requirement directly supports the overall project goal.

7. Modularity

The system should have a modular design, allowing each of the four components to function independently while still contributing to the larger integrated platform. This aligns well with the project structure, where the system call, user behavior, network traffic, and endpoint log models are separate yet complementary. The documentation outlines distinct pipelines, model flows and preprocessing stages for each component, indicating that modularity is necessary for maintenance and future integration.

8. Maintainability

The system must be maintainable, enabling updates to models, preprocessing pipelines, thresholds, and artifacts over time. This aligns with the endpoint component's use of serialized vectorizers, scalers, encoders, and model artifacts, as well as the system call component's saved Random Forest model, scaler, RNN checkpoint and training report. As cyber threats evolve, the project needs a

framework that allows for retraining, tuning, and updating without needing to rebuild the entire system from scratch.

9. Interpretability and analyst usability

The system should provide outputs that analysts can understand and use effectively. This requirement aligns with all four components, especially the endpoint and system call modules, which preserve session context, process chains, event sequences, and behavior details for review. Even with advanced machine learning models, their decisions must remain usable in security operations; otherwise, practical value will be limited. Thus, the system needs to produce interpretable outputs such as labels, session context, and clear threat indicators.

10. Robustness to heterogeneous data

The system must be capable of handling various data formats, structures, and security contexts across the four components. This requirement aligns with the project, as one component processes Windows endpoint logs, another handles system-call traces, a third manages user behavior sessions, and another examines network traffic patterns. The preprocessing and normalization pipelines in the documentation indicate that the system is designed to handle diverse cybersecurity data rather than just one standardized source.

11. Security and safe handling of telemetry

Since the system processes sensitive operational telemetry, like endpoint logs, command lines, process traces, user behavior data, and network activity, it must ensure secure handling of collected data and prediction outputs. While the uploaded components mainly focus on detection methodologies, this requirement spans all four modules, as each deal with sensitive information. In deployment, this means safeguarding logs, traces, models, and outputs from unauthorized access, in accordance with the sensitive nature of the data described in the component documents.

12. Availability for continuous monitoring

The system should remain available during ongoing monitoring and threat prediction activities. This is especially relevant for the real-time project goal and the live monitoring aspects of the system call, endpoint, and user behavior components. If the system is not continuously available, it may miss early signs of attack behavior. Therefore, continuous operational availability is an essential quality for the overall platform.[17]

2.7 Commercialization and Deployment Considerations

From a commercialization angle, this project has strong potential because it fills a clear industry gap. Organizations need systems that can go beyond simple signature matching and proactively predict both known and unknown threats. The project is especially important for companies that generate a lot of telemetry but struggle to turn those data streams into useful predictive security intelligence. Using endpoint logs, user behavior, network traffic, and system-call traces makes the proposed framework valuable for environments needing layered cyber defense. A commercially

deployable version of the system would likely be offered as an integrated cybersecurity analytics platform or as a modular security solution with separate deployable parts. For instance, an organization could deploy just the endpoint log model for Windows endpoint analytics or combine it with the user behavior and network anomaly modules for wider protection. Because the methodology is modular, it allows flexible product packaging. This would make commercialization more feasible than a strict one-model design.

In deployment terms, the project needs access to operational telemetry sources. For the endpoint component, Sysmon and PowerShell logs must be collected and normalized. For the system call component, live Sysmon monitoring and trace reconstruction must be in place. For the user behavior component, activity logs need to be grouped into temporal sessions and for the network component, network traffic must be captured and changed into features that fit the anomaly detection pipeline. Thus, deployment planning must include data connectors, ingestion agents, preprocessing services, and model-serving infrastructure.

Another key consideration in deployment is model lifecycle management. Since the system relies on trained artifacts such as vectorizers, scalers, encoders, Random Forest models, RNN checkpoints and other classifiers, deployment must include artifact storage, version control, and consistent inference pipelines. The system call component specifically notes that saved artifacts and scaling consistency are essential at inference time, and the endpoint component uses serialized vectorizers and encoders. This means commercialization would need reliable handling of model assets.

There are also important issues related to false alarms, operational trust and analyst adoption. Even if the system works well, deployment success hinges on whether analysts trust and use the outputs. This is why selecting thresholds, preserving context, reviewing at the session level, and creating meaningful classification categories are vital. Commercial deployment would benefit from adjustable thresholds, role-based alert views and feedback loops to calibrate models to organizational needs over time. This conclusion stems from the project's focus on alerting, thresholds and operational workflows.

2.8 Tools and Technologies

The project uses a mix of machine learning, deep learning, log analytics, and preprocessing technologies across its four components. In the network traffic component, the key technologies are Autoencoder deep learning, Isolation Forest, and logistic regression meta-classification. The Autoencoder learns normal network behavior by measuring reconstruction error, Isolation Forest isolates anomalies through random partitioning, and logistic regression combines the outputs to make a final prediction. In the user behavior component, the main technology is an LSTM-based deep learning architecture created for analyzing temporal sequences. The architecture starts with an input layer that represents 14 days with 24 features. This is followed by an LSTM layer with 128 units, dropout layers, dense layers with ReLU activation, and a sigmoid output for predicting

anomaly probability. This component also relies on engineered behavioral features from file activity, USB behavior, temporal usage, and cross-signal patterns.

In the endpoint log component, the main technologies are XGBoost, TF-IDF vectorization, StandardScaler, LabelEncoder, NumPy and Pandas. The model examines 369 features per sample, including TF-IDF vectors for process names, command-line arguments and event-chain sequences, along with 19 structured behavioral indicators. This component also employs regex-based detection of suspicious patterns for encoded commands, download strings, execution bypass patterns, and suspicious utilities.

In the system call component, the primary technologies are Sysmon-based collection, CAPEv2-style trace normalization, Random Forest and RNN sequence modeling. The pipeline also includes structured feature extraction, API vocabulary encoding, fixed-length sequence construction, probability-based routing and live monitoring endpoints for real-time analysis. These technologies are chosen to maintain a balance between speed, clarity and detailed sequential learning.

Tools

1. Visual Studio Code

Visual Studio Code served as the main development environment for writing, editing, debugging, and managing the project source code. It supported Python scripts, machine learning pipelines, backend logic, and integration tasks across various project components. Its extensions and debugging features made development more efficient.

2. Jupyter Notebook

Jupyter Notebook was used during the experimental and model development stages to test preprocessing steps, perform exploratory data analysis, train machine learning models, evaluate outputs, and visualize results. It was especially helpful for trying different model configurations and observing intermediate outputs.

3. GitHub

GitLab or GitHub was used for version control and managing source code. It helped organize project files, track code changes, support collaboration among team members and maintain different project versions during development. It also provided a clear way to store the project's implementation history.

4. Windows Sysmon

Sysmon acted as a system monitoring tool to collect detailed data on Windows endpoint and process activity. It provided important information such as process creation events, registry activity, DLL loading and other behaviors needed by the system call and endpoint log components.

5. PowerShell Logging

PowerShell logging tools captured script execution details and command activity on Windows systems. These logs were crucial for endpoint log analysis because they helped identify suspicious PowerShell behavior and commands related to attacks.

6. CAPEv2-style Trace Format

A CAPEv2-style trace structure was used in the system call component to normalize and represent behavioral traces consistently. This allowed the conversion of raw process activity into structured records suitable for feature extraction and sequential analysis.

7. Dashboard

A dashboard or monitoring interface displayed prediction results, alerts, session details, and security insights in a more understandable format. This supported workflows by making model outputs easier to interpret and review.

8. Operating System Environment

Windows was a key operating environment for collecting Sysmon and PowerShell logs, while Python-based development and machine learning execution happened in a standard software development environment. This setup accommodated both data collection and implementation needs.

Technologies

1. Python

Python served as the main programming language for the entire project. It was chosen for its strong support of machine learning, data preprocessing, log analysis, automation, and backend integration. All four project components depend heavily on Python.

2. Machine Learning

Machine learning was the primary technology of the project. It built predictive models that detected both known and zero-day cyber threats. Different machine learning approaches were used based on the problem type, including supervised learning, unsupervised learning, ensemble learning, and hybrid model design.

3. Deep Learning

Deep learning was applied in areas where complex behavioral and temporal patterns were important. This was particularly vital for the Autoencoder in the network traffic component and for the LSTM and RNN models in the user behavior and system call components. Deep learning enabled the system to uncover more complex hidden relationships in the data.

4. Pandas

Pandas were used for data manipulation, cleaning, transformation, and structured analysis. It supported tasks like reading datasets, handling log files, organizing feature tables, and preparing data for model training and testing.

5. NumPy

NumPy was utilized for numerical computation and array-based data processing. It played a key role in mathematical operations, vector management, model input preparation, and efficient manipulation of feature matrices.

6. TensorFlow / Keras

TensorFlow and Keras were used to implement deep learning models such as the Autoencoder, LSTM, and RNN architectures. These frameworks offered model-building, training, validation, and inference capabilities for components that required neural network analysis.

7. Scikit-learn

Scikit-learn was used to implement traditional machine learning algorithms and preprocessing methods. It supported models like Isolation Forest, Logistic Regression, Random Forest, and preprocessing utilities such as scaling, label encoding, and TF-IDF vectorization.

8. XGBoost

XGBoost was utilized in the endpoint log component for multiclass attack prediction. It was chosen for its strong performance with structured and high-dimensional data, especially when combining textual and numerical features.

9. TF-IDF Vectorization

TF-IDF was employed as a text representation technology in the endpoint log component. It converted process names, command-line arguments, and event sequences into numerical feature vectors, which allowed machine learning models to analyze important log content.

10. Sequence Modeling

Sequence modeling was an essential technology in the user behavior and system call components. It preserved the order of actions and identified suspicious patterns that developed over time instead of in isolated events.

11. Anomaly Detection

Anomaly detection was a key technology used in the network traffic component to identify zero-day threats. Instead of relying on labeled attack signatures, it learned normal behavior and flagged deviations as suspicious. This technology was crucial for spotting previously unseen cyber threats.

12. Feature Engineering

Feature engineering was used across all four components to convert raw security data into valuable model inputs. This included statistical features, behavioral indicators, sequential representations, anomaly scores, and textual encodings. It was a significant technology in the project.

13. Real-Time Data Processing

Real-time or near-real-time data processing was a vital technology requirement because the project aimed to support live cyber threat prediction. This allowed the system to process incoming data and generate predictions quickly enough for proactive defense.

14. Log Analysis and Behavioral Analytics

Log analysis and behavioral analytics are essential technologies used to examine endpoint activity, user actions, process traces, and network behavior. These technologies enabled the project to move beyond static signatures and focus instead on activity patterns and abnormal behavior.

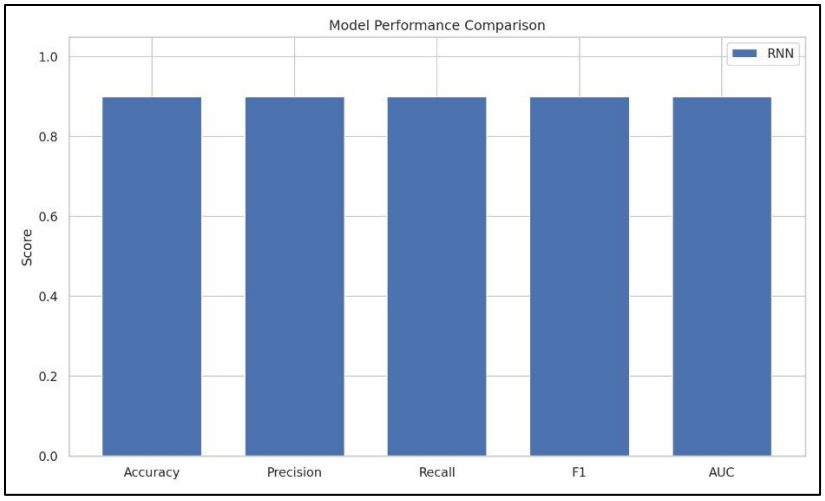
3. Results and Discussion

This section presents the experimental results of the main parts of the proposed Real Time Cyber Threats Predicting System. Each part focuses on a different aspect of cybersecurity; therefore, the results are shown separately for each part. This approach makes it easier to see the contribution and performance of each model within the overall project.

Known Attack Prediction by Analyzing System Call Patterns

The System Call Component added value to the project by examining process-level behavior traces and aiding malware prediction with a staged decision setup. According to the earlier model performance chart, the RNN-based system call model performed well in accuracy, precision, recall, F1-score and AUC, with all values near 0.90. This shows that the system call model effectively learned important sequential behavior patterns from process activity and provided dependable prediction results.

These findings suggest that system call traces hold crucial sequential information for spotting malicious process behavior. This component keeps the order of actions instead of relying solely on static summaries, which adds an important behavioral dimension to the project. Thus, the System Call Component enhanced the overall framework by improving process-level predictive ability.



Predicting Known Insider Attacks Using User Behavioral Patterns

The User Behavior Component was assessed using a classification model to differentiate between benign and malicious user actions. As shown in the uploaded classification report image, the model delivered perfect results across all evaluation metrics. The outcomes show 1.00 precision, 1.00 recall, and 1.00 F1-score for both the Benign and Malicious categories, with an overall accuracy of 1.00 on 98,904 samples.

Specifically, the model accurately classified 47,638 benign samples and 51,266 malicious samples while maintaining perfect macro-average and weighted-average values of 1.00. These results indicate that the user behavior model was highly effective in recognizing behavioral patterns and distinguishing normal user actions from suspicious or harmful behavior. This shows that the chosen feature representation and behavioral learning method suited this component very well.

These results matter because user behavior data can be intricate and hard to categorize due to the similarities between legitimate and harmful actions. Nonetheless, the reported results demonstrate that the model captured these differences accurately. Therefore, the User Behavior Component made a significant contribution to the project by offering reliable behavior-based threat detection.

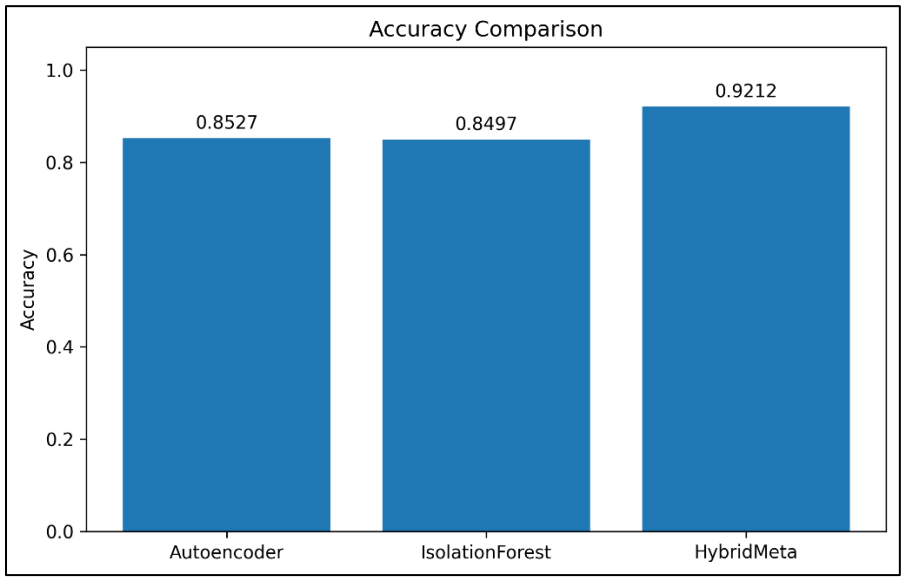
```
=== Classification Report ===
              precision    recall  f1-score   support

      Benign       1.00      1.00      1.00     47638
   Malicious       1.00      1.00      1.00     51266

    accuracy                           1.00     98904
   macro avg       1.00      1.00      1.00     98904
weighted avg       1.00      1.00      1.00     98904
```

Predicting Zero-Day Attacks Using Network Traffic Patterns

The Network Traffic Component aimed to detect zero-day attacks by examining unusual traffic behavior using anomaly detection models. Based on the uploaded accuracy comparison chart, the Autoencoder reached an accuracy of 0.8527 (85.27%), the Isolation Forest got 0.8497 (84.97%), and the Hybrid Meta model achieved 0.9212 (92.12%).

These results highlight that the Hybrid Meta model clearly outperformed the two individual models. This means that merging the outputs of the Autoencoder and Isolation Forest into a hybrid framework produced a more effective final prediction model. The result is particularly important for zero-day attack detection since unknown threats are hard to identify with rule-based methods. By better understanding normal traffic patterns and spotting deviations, the hybrid model improved the detection ability of this component.

Therefore, the network traffic results confirm that the hybrid anomaly detection method was the most effective for this component and significantly boosted zero-day detection performance compared to using standalone anomaly detection models.



Known Attack Prediction by Analyzing Endpoint Log Patterns

The Windows Endpoint Log Component was assessed using a multi-class classification model to predict known behaviors from endpoint telemetry. As indicated in the uploaded classification report image, the model achieved an overall accuracy of 0.9028 (90.28%) on 41,524 samples. The weighted average precision stood at 0.9339, the weighted average recall at 0.9028, and the weighted average F1-score at 0.9064.

At the class level, the model excelled for the benign class, attaining 1.0000 precision, 0.9891 recall, and 0.9945 F1-score. The credential_dumping class also performed well, with 0.9738 precision,

0.8090 recall, and 0.8838 F1-score. The lateral_movement class had a very high recall of 0.9903, though its precision was lower at 0.4848, indicating that the model was very sensitive in detecting this class but also made some misclassifications. The defense_evasion and privilege_escalation classes had lower recall values of 0.5635 and 0.5628, suggesting these classes were more challenging to distinguish due to overlapping behavior patterns.

```
Classification Report
                                                                    
                      precision    recall  f1-score   support

              benign     1.0000    0.9891    0.9945     28186
  credential_dumping     0.9738    0.8090    0.8838      3172
     defense_evasion     0.8826    0.5635    0.6879      3895
    lateral_movement     0.4848    0.9903    0.6510      3085
privilege_escalation     0.8073    0.5628    0.6632      3186

            accuracy                         0.9028     41524
           macro avg     0.8297    0.7829    0.7761     41524
        weighted avg     0.9339    0.9028    0.9064     41524
```

Overall, these results show that the endpoint log model was effective in predicting known attack types from Windows Sysmon and PowerShell log data. The strong performance in detecting benign and lateral movement actions demonstrates the usefulness of endpoint logs for attack classification, while the lower performance for certain classes highlights the challenges in distinguishing behaviorally similar attack types. This component thus provided solid host-level attack prediction capabilities for the overall system.

3.1 Research Finding

The findings of this project show that predicting cyber threats is more effective when multiple security data sources are analyzed together instead of relying on just one detection method. The overall project confirmed that meaningful threat indicators can be identified from system call traces, user behavior data, network traffic patterns, and Windows endpoint logs. Each component addressed a different threat perspective. Together, they demonstrated that a layered machine learning approach is better for predicting both known attacks and zero-day threats than traditional signature-based systems alone.

A key finding is that behavior-based analysis offers stronger predictive capabilities than static rule-based detection. Across all four components, the project showed that cyber threats often manifest as abnormal behavior, suspicious event chains, temporal deviations, or unusual activity patterns rather than fixed signatures. This suggests that predictive cybersecurity systems need to learn behavioral patterns from data rather than just relying on previously known attack definitions.

Another important finding is that different types of cyber data require different modeling strategies. The results indicated that sequence-based models worked better for user behavior and

system calls, hybrid anomaly detection was more suitable for network traffic, and multiclass classification excelled in analyzing structured endpoint logs. This confirms that no single algorithm can solve all cybersecurity problems and that using specialized models for each component improves overall prediction capabilities.

Known Attack Prediction by Analyzing System Call Patterns

The system call component found that process-level behavior contains both statistical and sequential patterns that are useful for threat prediction. By converting raw process activity into structured traces, the component generated both aggregate Random Forest features and ordered RNN sequences. This showed that malware-related behavior is better analyzed when summary information and action order are preserved.

Another finding from this component is that the two-stage model design improves efficiency and practicality. The Random Forest quickly handled clear benign or malicious cases, while the RNN focused only on uncertain or grey-zone traces. This supports the idea that layered decision-making can cut processing costs while still enabling deeper analysis when necessary.

Predicting Known Insider Attacks Using User Behavioral Patterns

The user behavior component found that temporal analysis is very effective for spotting insider threats and abnormal user actions. The LSTM-based UEBA model used 14-day activity sequences and 24 engineered behavioral features, allowing it to identify suspicious changes in file access, USB activity, and time-based behavior. This indicated that user-related threats are better detected through patterns over time instead of single isolated events.

The experimental results were very strong, with 94.3% accuracy, 94.5% precision, 91.3% recall, 92.8% F1-score, and 0.946 AUC-ROC. These results show that sequence-based deep learning is very suitable for detecting behavioral anomalies and can provide reliable performance for real-world insider threat monitoring.

Predicting Zero-Day Attacks Using Network Traffic Patterns

The network traffic component found that zero-day threats can be detected by learning normal traffic behavior and spotting deviations from it. This is significant because zero-day attacks do not have predefined signatures, so traditional detection methods often fail. The study confirmed that anomaly detection is an effective way to identify previously unseen malicious activity in network traffic.

A major finding from this component is that the hybrid model outperformed the individual models. The hybrid approach, which combined Autoencoder, Isolation Forest, and logistic regression, achieved 92.12% accuracy, surpassing the standalone Autoencoder at 85.27% and Isolation Forest at 84.97%. This showed that combining complementary anomaly signals leads to better zero-day detection performance than relying on a single unsupervised model.

Known Attack Prediction by Analyzing Endpoint Log Patterns

The endpoint log component found that Windows endpoint telemetry is a strong resource for predicting known attacks, especially when logs are analyzed in contextual sessions rather than as

individual events. By using Sysmon and PowerShell logs, the component captured process behavior, command-line semantics, event chains and suspicious execution patterns in a structured feature space. This showed that endpoint logs are valuable not only for forensic review but also for predictive cyber defense.

The XGBoost classifier in this component achieved 90.28% accuracy and 0.9064 weighted F1-score, with particularly strong results for benign activity and lateral movement detection. However, some classes, such as defense evasion and privilege escalation, were more challenging to distinguish because of behavioral overlap. This suggests that while endpoint prediction is highly effective, some attack categories share similar patterns and remain more difficult to classify.[18]

3.2 Overall Components Findings

Considering the results of all four components together, the project reveals that different threat types become visible at different layers of digital activity. System calls capture process-level behavior, user analytics reveal insider and misuse patterns, network traffic supports zero-day anomaly detection and endpoint logs aid in classifying known attacks. This emphasizes that no single data source can provide complete predictive coverage on its own.

The final research finding of the project is that a multi-layered, machine learning-driven framework offers a stronger foundation for proactive cybersecurity defense. By integrating the strengths of all four components, the project shows that it is possible to predict both known and zero-day threats more effectively, while also enhancing the practical usefulness of cyber threat intelligence for real-time security monitoring and response.

4.Summary of Each Student's contribution

Student	InvisiGuard - Intelligent System for Predicting Cyber Threats Real Time	Summary of Contribution
Nawarathna.N.M.I.M IT22343116	Known Attack Prediction by Analyzing System Call Patterns	The contribution involved collecting and processing activity at the process level, converting raw Sysmon events into structured behavioral traces, extracting both aggregate and sequential features, and supporting a two-stage prediction flow using a Random Forest model and an RNN fallback model. This component helped the overall system by enabling process-level threat detection based on system behavior patterns.
Pallewaththa P.A IT22133304	Predicting Known Insider Attacks Using User Behavioral Patterns	The contribution also involved designing the LSTM-based User and Entity Behavior Analytics model, preparing the behavioral dataset, engineering 24 user activity features, and training the model to detect suspicious and unusual user actions over time. This component strengthened the system by providing detection of insider threats and abnormal user behavior using temporal analysis.
Nawarathna N P D T IT22117496	Predicting Zero-Day Attacks Using Network Traffic Patterns	The contribution included developing a hybrid anomaly detection framework using an Autoencoder and Isolation Forest. It also involved preparing traffic features, training the models, and evaluating the performance of anomaly detection. This component improved the overall system by enabling the detection of unknown and previously unseen network-based cyber threats.
Nethkalum G M H IT22562906	Known Attack Prediction by Analyzing Endpoint Log Patterns	The contribution consisted of collecting and normalizing Sysmon and PowerShell logs, grouping endpoint events into sessions, engineering textual and numerical features, and building the XGBoost multiclass prediction model for known attack categories. This component enhanced the project by enabling the prediction of known attacks,

		such as credential dumping, defense evasion, lateral movement and privilege escalation from Windows endpoint telemetry.

5.Conclusion

In conclusion, this project successfully showed the design and development of a Real Time Cyber Threats Predicting System that tackles both known attacks and zero-day threats using a multi-component machine learning framework. Unlike traditional cybersecurity systems that mainly rely on signature-based detection, this project demonstrated that predictive security can greatly improve by analyzing various behavioral data sources, such as system call traces, user behavior patterns, network traffic patterns and Windows endpoint logs. Each component offered a unique detection perspective. Together, they formed a broader, more proactive and intelligence-driven cybersecurity solution.

The project findings confirmed that machine learning and deep learning methods are effective at identifying suspicious behaviors that static rules alone cannot easily detect. The system call component highlighted the benefits of combining summary-based and sequential process behavior analysis. The user behavior component showed that temporal modeling can effectively spot insider threats and unusual user activity. The network traffic component proved that hybrid anomaly detection can enhance zero-day threat prediction, while the endpoint log component demonstrated that Windows telemetry can effectively classify known attack patterns. These results confirm that different cyber threats become apparent at various layers of system activity. An integrated framework is more effective than relying on a single-source detection model.

The project also emphasized the importance of using specialized models for different data sources. Sequence-based models worked best for user behavior and system calls, hybrid anomaly detection was more effective for network traffic, and multiclass classification excelled in endpoint log analysis. This indicates that predicting cyber threats is not a one-model problem. Instead, it requires various analytical strategies based on the structure and meaning of the underlying data. Thus, the project makes an important contribution by showing how these different methods can exist within one unified research direction. Another key takeaway is that the proposed project has solid practical relevance. Several components were designed with real-time or near-real-time workflows in mind, including live monitoring support, near-real-time endpoint analysis, and low-latency behavioral inference. This suggests that the project goes beyond theoretical model experimentation. It also lays a foundation for deployment-oriented cybersecurity monitoring and alerting systems. Therefore, the research contributes not only to academic knowledge but also to developing more practical and proactive cyber defense solutions.

Overall, this project achieved its main goal of proposing an integrated predictive cybersecurity framework that can improve detection of both known and unknown attacks. The combined results of all four components show that a multi-layered, behavior-based, and machine learning-driven

approach can strengthen cyber threat intelligence, enhance proactive defense capabilities and provide a significant step beyond traditional signature-based security methods. As a result, the project establishes a solid basis for the future development of intelligent cybersecurity systems that are flexible, scalable and better prepared to handle emerging threats.

6. References

[1] M. Carannante, "An Analytical Review of Cyber Risk Management and Cyber Insurance," Risks, vol. 13, no. 8, 2025.

[2] R. Pal, "Cyber-Insurance in Internet Security: A Dig into the Information Asymmetry Problem," arXiv preprint arXiv:1202.0884, 2012.

[3] W. Dou, "An insurance theory based optimal cyber-insurance contract model," Information Sciences, vol. 512, pp. 1–15, 2020.

[4] T. J. Boonen, "Cybersecurity investments and cyber insurance purchases in a non-cooperative game," ASTIN Bulletin, 2025.

[5] U. Franke, "Interdependent cyber risk and the role of insurers in facilitating security investments," Computers & Security, 2025.

[6] G. Zeller et al., "Risk mitigation services in cyber insurance: optimal contract design," The Geneva Papers on Risk and Insurance, 2023.

[7] E. Kopp, L. Kaffenberger, and C. Wilson, "Cyber Risk, Market Failures, and Financial Stability," International Monetary Fund (IMF), 2017.

[8] H. R. K. Skeoch, "The barriers to sustainable risk transfer in the cyber-insurance market," Cybersecurity, vol. 10, 2024.

[9] Y. Gómez, "Cyberinsurance adoption strategies and cybersecurity behavior," Behaviour & Information Technology, 2025.

[10] S. Liu and Q. Zhu, "Mitigating Moral Hazard in Cyber Insurance Using Risk Preference Design," IEEE/ACM Transactions, 2022.

[11] M. M. Khalili, P. Naghizadeh, and M. Liu, "Designing Cyber Insurance Policies: Mitigating Moral Hazard Through Security Pre-Screening," 2017.

[12] J. L. Vagle, "Cybersecurity and Moral Hazard," Stanford Law Review, 2020.

[13] "Asymmetric Information," ScienceDirect Topics, Elsevier.

[14] M. Carannante et al., "Cyber risk quantification and insurance pricing models," Risks, 2025.

[15] M. V. Pauly, "Moral Hazard and Adverse Selection in Insurance Markets," International Journal of the Economics of Business, vol. 31, no. 3, 2024.

[16] OECD, "Encouraging Clarity in Cyber Insurance Coverage," Organisation for Economic Co-operation and Development, 2023.

[17] S. Liu, “Mitigating moral hazard in insurance contracts using risk preference design,” Insurance: Mathematics and Economics, 2025.

[18] J. Florackis et al., “Market impact of major cyber-attacks on insurers,” Journal of Financial Markets, 2024.